Tor Vergata University of Rome

FACULTY OF ENGINEERING

COMPUTER SCIENCE, CONTROL AND GEOINFORMATION
CYCLE XXXVII

PhD Thesis

A UNIFIED SOFTWARE ARCHITECTURE FOR DISTRIBUTED AUTONOMOUS SYSTEMS

ADVISOR

Prof. Sergio Galeani

CANDIDATE

Roberto Masocco

COORDINATOR

Prof. Francesco Quaglia

A.Y. 2023/2024

TOR VERGATA

UNIVERSITÀ DEGLI STUDI DI ROMA

COMPUTER SCIENCE, CONTROL AND GEOINFORMATION

CYCLE XXXVII

A UNIFIED SOFTWARE ARCHITECTURE FOR DISTRIBUTED AUTONOMOUS SYSTEMS

PHD THESIS

Advisor:
Prof. Sergio Galeani Signature:_____

Coordinator:
Prof. Francesco Quaglia Signature:_____

Candidate:
Roberto Masocco Signature:_____

A.Y. 2023/2024

Tor Vergata University of Rome, Faculty of Engineering

Via del Politecnico 1, Rome, Italy

Wings are a constraint that makes
it possible to fly.
— Robert Bringhurst

Acknowledgements

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Lausanne, 12 Mars 2011

Preface

This is the summary of three years of work spanning multiple areas. From the problem to the solution. It is inevitable to name some tools and concepts before they are explained later on among chapters.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis.

Pellentesque cursus luctus mauris.

Contents

Acknowledgements i

Preface ii

List of Figures vi

List of Tables vii

Listings viii

1 Introduction 1

2 The importance of an architecture 6
2.1 Design challenges . . . 8
2.1.1 The tasks . . . 8
2.1.2 The hardware . . . 9
2.1.3 The software . . . 11
2.1.4 The data . . . 12
2.1.5 The testing . . . 12
2.2 Fundamental requirements . . . 13
2.3 Related work . . . 17
2.3.1 RoboStack . . . 18
2.3.2 eProsima Vulcanexus . . . 20
2.3.3 NASA F Prime . . . 21
2.3.4 MARTe2 . . . 22

3 Tools for the job 26
3.1 Inter-process communication: the middleware . . . 27
3.1.1 Data Distribution Service . . . 32
3.1.2 Zenoh . . . 39
3.1.3 Robot Operating System 2 . . . 46
3.2 Version control . . . 69
3.2.1 Git . . . 73
3.3 Containerization . . . 79
3.3.1 Docker Engine . . . 84

Bibliography **96**

List of Figures

2.1 Organization of the core libraries of the MARTe2 framework. 23

3.1 Software organization in a general-purpose computer, as presented at the 1968 NATO conference on software engineering. 30
3.2 A network of applications using the DDS, each denoted by its own `DomainParticipant` [13]. 36
3.3 Possible network topologies in Zenoh [14]. 45
3.4 Placement of the Zenoh protocol in the network stack [14]. 46
3.5 ROS 2 logo. 48
3.6 Scheme of the message communication primitive [33]. 55
3.7 Scheme of the service communication primitive [34]. 55
3.8 Scheme of the action communication primitive [35]. 56
3.9 Core ROS 2 API organization [36]. 59
3.10 ROS 2 event-based execution model. 61
3.11 Simulation of an autonomous drone with imaging and depth sensors in Gazebo (right) and visualization of the data in RViz (left). 65
3.12 micro-ROS architecture [62]. 69

List of Tables

Listings

3.1 Definition of the `sensor_msgs/Image` message. 56
3.2 Definition of the `example_interfaces/AddTwoInts` service. 57
3.3 Definition of the `example_interfaces/Fibonacci` action. 57

1 Introduction

In the history of humanity, many turning points have been marked by technological advancements. From the invention of the wheel to the steam engine, from the light bulb to the computer, technology has continuously changed how we live, work, and see the world. Its impact on society has always been profound and sometimes positive, sometimes negative. The changes it brought affected our daily habits, introducing new ways of doing things and sometimes even new ways of thinking. With the same spirit that drove our ancestors to develop *tools* made of stones and sticks to make their lives easier while harvesting food, we are now developing ever more powerful machines of all sorts to handle the challenges of the modern world, which can work either together with a human operator or in complete autonomy. Moreover, considering the recent developments of artificial intelligence, these tools have reached such a level of sophistication that they may soon reshape our thought processes.

The technological advancements that brought us these new solutions increased not only their capabilities but also their complexity and, ultimately, their variety. An excellent example is the industrial manipulator robot: a machine that can perform various tasks, from simple pick-and-place operations to more complex ones such as welding, painting, or even surgery. These machines have evolved from mechanical arms composed of multiple actuators, all connected to a single power and computational unit, to a *cyber-physical system*: a network of sensors and microcontrollers, grouped

in subsystems all interconnected to a central unit, which is, in turn, responsible for their coordination, supervision, and control, as well as for interfacing with human operators. In this case, this evolution has been driven by the need to increase the robot's capabilities, making it more versatile and adaptable to different tasks, and by the need to make it more autonomous as well as more reliable, reducing the risk of accidents and downtime due to maintenance. Another suitable example would be the modern airplane: considering even a small civilian aircraft, originally equipped with an engine, control surfaces, and a few instruments, it has evolved into a complex system of systems composed of a multitude of sensors, probes, indicators, and avionics. In this scenario, the evolution has been driven by specific needs still related to safety and comfort but can ultimately be summarized in the need to ease the lives of both the passengers and the crew.

This design and development paradigm has recently become commonly denoted by the adjective *distributed*. The definitive characteristic of a distributed system is its decomposition into a network of subsystems. Each one is responsible for a specific task; all of them are interconnected, and they all perform a piece of the job that the entire macroscopic system has been designed to do. This decomposition allows for more efficient use of the resources, better scalability, and higher reliability, as the failure of a single subsystem does not necessarily lead to the failure of the whole system. Moreover, the distribution of the subsystems allows for a better organization of the system's functionalities, as each subsystem can be designed and developed independently from the others, following a modular approach that eases the task of understanding, controlling, and maintaining the system as a whole. This approach has been widely adopted in many fields, from industrial automation to aerospace, automotive to robotics, telecommunications to energy. Another peculiar characteristic of a distributed system is the possibility, in many contexts, to deploy its parts

in different locations: coming back to the manipulator example, it is common to have the sensors and the actuators distributed along the robot's arm and the control unit placed in a separate location, while in the airplane example, the sensors and the avionics are distributed along the fuselage and the wings, while the cockpit is placed in the front of the aircraft; in pure software systems, the subsystems can be deployed in different servers, data centers, or even continents, with the additional advantage of partitioning the workload generated by the various regions among local computational nodes. These degrees of physical distribution allow great flexibility in design and resource allocation, but they also introduce new challenges in communication, synchronization, and fault tolerance.

All of this innovation comes at a price: when the multitude of subsystems, subcomponents, and submodules increases, new problems arise. The system's complexity grows, and so does the difficulty of understanding, controlling, and maintaining it. The interactions between the different parts of the system become intricate. With strong dependencies among modules and no redundancy, the consequences of a failure in one of them can propagate to the others, causing a cascade of failures that can lead to catastrophic events. A deep understanding is needed to prevent events like these, for which a good organization of the system *architecture* is of paramount importance. Moreover, a well-designed architecture may also be incredibly beneficial in two other stages: the design and development of the system and its maintenance. In the former, a clear and well-structured architecture naturally leads to a clear and well-structured design, following a top-down approach that starts from the system's requirements and ends with the definition of the subsystems and their interfaces, each following a common set of standards and best practices that adhere to the structure of the architecture itself. In the latter, a well-structured architecture allows for a clear identification of the subsystems and their interactions, easing diagnosing and fixing a

failure or an error. Finally, a good architecture can also be a powerful resource for the evolution of the system as a whole, allowing for the addition of new functionalities or the replacement of old components with new ones without the need to redesign the whole system from scratch, significantly reducing the time and the costs of such an operation.

The subject of this work is the development of a software architecture for distributed systems intended to control dynamic processes and autonomous agents. This setting may seem specific, but it is pretty broad: all of the above examples fall into these categories, representing topics of high interest in today's research and industrial environments. All the design and development problems listed above still apply to these cases, which also bring their own specific challenges addressed in the following. The main goal of this work is to provide a unified solution to these problems by defining a software architecture that can be used as a reference for the design and development of distributed systems in the contexts mentioned above. The proposed architecture is designed to be modular and flexible, and it is based on a set of standards and best practices that may guide the development of the subsystems and their interactions. Moreover, the architecture is designed to be adaptable to and deployable on different hardware platforms, to provide an abstraction layer for solutions ranging from embedded systems to general-purpose computers, and to easily integrate other software modules like third-party libraries. Finally, the architecture is built to be easily usable by developers, providing a unified set of tools and utilities that ease the task of developing, testing, and deploying new modules for the classes of systems mentioned above. The end goal of this work comes from a fundamental necessity identified by the author during his experience as a researcher and developer in the field of automation and robotics: having a standard development environment suitable for a single developer or an entire team, capable of replicating almost entirely the system configuration of a

target platform on a development machine and enabling distributed applications to work and behave in the same way on all supported platforms, from build to run time. This way, the integration overhead gets dramatically cut, allowing developers to focus on their most essential and only true task: research and development, leading quickly to new technological advancements.

Just like our primordial ancestors faced the necessity of building tools to ease their lives, this work follows the same approach: identify the problem, lay down the requirements, design the solution, build it, test it, and finally use it. The requirements that drive the design of the architecture are identified and discussed in the following Chapters, and the proposed solutions are based on the state of the art in the field of software engineering and distributed systems, as well as on the author's own practical experience.

2 The importance of an architecture

The development of an autonomous system is a process that involves various stages. Each stage is typically characterized by defining and solving issues related to different aspects of the system, ranging from the tasks it will be required to perform, to its construction and programming, and even to how the data it collects and that affect its operation should be presented to potential operators. Generalizing from a specific application case, most issues can be classified into a few high-level categories. Although it may seem counterintuitive from the division that follows, these categories are not always addressed sequentially, nor are they entirely independent of each other. To address situations like these, various project management techniques have been developed in recent times, aimed at minimizing the risk that an error made in an earlier phase could compromise subsequent stages, potentially undermining finished work and increasing development times. In light of the observations made in Chapter 1, it is clear that, with the increasing complexity of modern automated systems and of the tasks they must perform, these topics are more relevant than ever. In particular, new tools made available to developers and designers will need to reflect these specificities, simplify the management of typical situations where possible, and highlight potential critical issues to aid in problem-solving. Practical experience shows that, despite the variety of cases and applications, the challenges faced are almost always, if not identical, very similar, as are the strategies to address them.

The aim of this chapter is to briefly present the issues typically encountered when designing an autonomous system, highlighting their unique characteristics without focusing on a particular application area, and then to examine some recent hardware and software solutions that enable the optimization of design, development, and testing activities. The discussion starts at a relatively high level of abstraction but becomes progressively more specific, forming the guiding thread that connects all subsequent chapters, finally highlighting the necessity of a well-structured software architecture to address the issues raised. The requirements that such an architecture must meet are then formalized and discussed in due detail.

Before moving further, it is instrumental to clarify the definition of an autonomous system in the context of this work. First of all, in this work the term *system* identifies a macroscopic entity, which can either be entirely hardware, or entirely software, or a mixture of both, that evolves in time and is characterized by a set of inputs and outputs. Its tasks are defined by the operations that it must perform on these inputs to produce the desired outputs, and by the constraints that these operations must satisfy. The system can either be seen as a unitary entity or as a set of *subsystems*, *subcomponents*, and *submodules*, all terms that are used interchangeably in this work, each of which takes care of a part of the processing, thus contributing to a part of the tasks. For the system to work correctly, all the subsystems must do the same and be interconnected accordingly, both in terms of the data they share and of the resources they use. The term *autonomous* refers to the capability of the system to perform these tasks without human intervention, or with minimal human intervention, once it has been set up and started. This definition is intentionally broad and generic, as it is meant to encompass a wide range of applications and scenarios, *e.g.*, from mobile robots to drones, from power plants to industrial automation systems. Another class of systems that is included in this definition and that is of interest in this work is

that of *dynamic systems*, which are systems whose state evolves in time according to a mathematical model that can be either deterministic or stochastic, and that can be either known or unknown. The evolution of the system's state, and thus that of its output, can be influenced by acting accordingly on its input, an operation that is commonly referred to as *controlling* the system or *applying a control law* to the system. In the context of autonomous systems this can be one of the tasks to perform, which is especially true for those systems that have a physical structure that can be modeled as a dynamic system and that should evolve over time according to a specific desired behavior.

2.1 Design challenges

2.1.1 The tasks

The initial decisions in the design of an autonomous system naturally concern the specifications of the operations it will be required to perform. These specifications must be fully defined from the onset, down to the smallest detail possible at this stage, as they will guide all subsequent choices, from hardware selection and construction to programming. During the breakdown of operational requirements into *tasks*, intended as discrete units of work to be executed, various types of these will emerge. Without delving, for now, into the specifics of a particular scenario, common categories of tasks include:

- **Continuous tasks**, which begin at system startup and run uninterrupted until it is turned off or malfunctions.
- **Periodic tasks**, which are performed at regular intervals.

- **Asynchronous tasks**, which are executed only when a specific event occurs, which may or may not ever occur.

It is easy to see that most functionalities in modern autonomous systems can be broken down into tasks that fall within these categories. These include operations ranging from actuator control, to the collection of data and measurements required by control algorithms, to communication with eventual operators and users. The characteristics of these tasks will influence both the hardware that must execute them and the software that encodes them, which should be organized accordingly to facilitate development and maintenance. To allow this organization, the system architecture must efficiently support the development of software modules, and the integration of hardware solutions, that can handle all these kinds of tasks.

2.1.2 The hardware

The next decisions to be made concern the hardware to be used. The choices made initially in this direction may very well not be definitive, since they would concern both the construction of the initial prototypes and the final product: there will likely be misjudgments or unforeseen issues to address down the road. Nevertheless, the hardware must be selected carefully based on the operational, structural, and construction requirements defined so far. Decisions taken in the previous step are of critical importance at this stage: establishing the computational complexity of the operations the system will need to perform will dictate the level of power required from the hardware, which may need to be equipped with discrete coprocessors[I] to handle specific types of data or subdivided into multiple interconnected subsystems.

[I] The term *discrete* here refers to auxiliary hardware installed in a computer that cannot operate independently but is designed to be driven by the primary computer, providing specialized support for specific tasks; classic examples include floating-point math coprocessors, parallel and graphic coprocessors, and sound cards.

A market survey is essential to assess whether pre-tested and ready-to-use components or SoCs[II] could be utilized rather than resorting to a proprietary solution, which generally entails significant development, testing, and validation costs. For the reasons mentioned above, it is very challenging, if not impossible, to outline a comprehensive and definitive list of possible hardware solutions at this point; the priority is to understand what is needed and whether the market offers options compatible with those requirements, while also anticipating potential future demands. Following this approach will allow any subsequent modifications to be managed with minimal effort.

The remaining choices concern lower-level hardware, which generally consists of:

- high-speed microcontrollers;
- actuators;
- sensors;
- interfaces and communication devices;
- data storage systems.

Nothing more can be said about these elements without reference to a specific case; however, practical scenarios discussed later in this work will follow this design pattern. From the multiplicity of devices mentioned up to this point, and that may compose a modern autonomous system, it starts to become clear how the complexity of the system can grow rapidly, as anticipated in Chapter 1. The need for a well-structured architecture to manage this complexity becomes more evident.

[II] System-on-Chip.

2.1.3 The software

Except for those components that perform crucial and highly specific functions, such as electrical or electronic circuits, nearly all operations carried out by a modern autonomous system are today encoded within an internal computer, which translates these instructions into commands for other components, subsystems, sensors, or actuators. This is not the place to delve into standard software design and development procedures, but it is worth reflecting on one point which has become quite common in present solutions: given the underlying complexity of the system itself, the software that controls it will also have a layered and highly hierarchical organization. In this hierarchy, the lower levels are occupied by modules that must interact with, if not directly control, the smaller and faster subsystems (*e.g.*, firmware), and these modules must be developed with particular attention to their characteristics and purposes. As we move up the hierarchy, we encounter broader modules that encode operations such as supervision, planning, and user communication, making them closer to the notion of *application* or *system software* which end users are more accustomed to as it is the most visible to them.

This context also involves decisions regarding the programming languages to be adopted. As is well known, each language has its unique features, highlighting its strengths as well as weaknesses compared to alternatives. Only a few programming languages are truly suitable for multiple purposes, in the sense of the ease with which they allow different types of tasks to be coded, and none should be considered universal.

When operating in this context, it is essential to rely on tools that simplify design and development as much as possible at all levels, assisting with the management of issues related to module organization, inter- and intra-process communication, data

processing, and storage of various data types. The necessity of a good architecture to handle these new challenges becomes increasingly evident.

2.1.4 The data

For an autonomous system to successfully complete its tasks, it requires a nearly constant stream of information. This information can be received in data packets managed by suitable communication interfaces, or it can be acquired directly through measurement devices, commonly known as *sensors*. For both solutions, typical challenges that arise immediately include the integration of these devices with the rest of the apparatus and the processing of the data they produce. These issues are crucial, as the operation of the entire system is almost always contingent upon obtaining information about measurable quantities of interest and the system's own state[III].

It is therefore reasonable to expect that the hardware architecture of a modern autonomous system will comprise numerous communication interfaces and sensors, the latter consisting of specific hardware designed to measure particular physical quantities of interest. The challenges mentioned above pertain to the integration of such hardware with the rest of the system and to the development of software modules capable of managing it, receiving and processing the data it generates. Both the hardware platforms and software tools should, as far as possible, be developed or selected with these requirements in mind, to minimize their integration overhead.

2.1.5 The testing

As soon as the initial prototypes are finalized, testing should begin to verify their functionality and confirm the validity of the design decisions made up to that point.

[III] Consider, *e.g.*, feedback control algorithms.

Bearing in mind that a generic and universal testing procedure cannot be defined for all types of autonomous systems, it can generally be expected that once hardware functionality is confirmed, attention will shift to software validation. Debugging software running on an autonomous system is generally more challenging than debugging common application software due to several specific factors:

- It is more difficult to trace program execution on an autonomous, potentially mobile device: direct access to the system may not be available, and the tools at hand are often limited or not very sophisticated.
- In the event of issues, both the software and hardware in use must be re-checked, as the problem could stem from hardware faults or incorrect use of the hardware.
- Conducting tests is not as simple as running a program and reviewing its results: the safety of the apparatus and its operators must be ensured throughout the entire procedure, which can make even the organization of a test a challenging task.

Given the difficulty and importance of this phase, it is reasonable to employ tools and methodologies that can facilitate these tasks as much as possible, ideally those that have already been adequately validated.

2.2 Fundamental requirements

The challenges outlined in the previous section are not exhaustive, nor are they entirely independent of each other. They are, however, representative of the issues that typically arise when designing an autonomous system. Summarizing briefly the many aspects previously covered, suppose that the task at hand is that of developing

an autonomous system capable of performing a series of operations. It is reasonable, in the context of this work, to focus on situations that meet the following assumptions:

- The system has a physical structure that can be modeled as a dynamic system, thus, one of the tasks consists in stabilizing and controlling this dynamic system, something that is commonly referred to in the following as *low-level control* or simply *control*.
- Digital programmable computers of different kinds are used to both develop and run all the algorithms, ranging from low-level control to task planning and supervision.

Note that the situations hereby described might span from mobile robots to drones and even power plants, all classes of systems that have been subject of the author's research work. Recalling Section 2.1, a series of choices regarding the following items must be made:

- sensors and actuators to use;
- software tools and packages to employ and integrate;
- hardware platforms, *i.e.*, onboard electronics to adopt.

Note how these items might be strongly interconnected. Low-level hardware must be chosen according to the control objective for the underlying dynamic system, while high-level hardware must be selected usually weighing the software that it will run against the physical capabilities, as well as the costs, of the system that is going to host it. Finally, software tools and packages can be divided in two groups: that which has to be developed from scratch, possibly very specific to the system at hand, and that

which can be reused from existing libraries or frameworks. The latter category, for example, includes solutions for data processing, communication, and visualization, as well as remote access and monitoring tools; when one wants to develop the prototype of an autonomous system, especially for research purposes, these things are often overlooked, but they are crucial for the system to be usable and maintainable and the overhead that they might introduce can easily become significant if not properly managed early on.

There is a principle in Computer Science [1], dating back to the very early days of the discipline, stating that to do a proper job *no problem should ever be solved twice* or, in a historical fashion, *one must never reinvent the wheel* with the only exception of building a better wheel. One can see how this fits perfectly in the context of the development of autonomous systems: there are a wide range of complex foundational and integration problems to solve which concern the very base of any such system, and thus present themselves each time a new one is developed. Since the author has spent a significant amount of time and effort solving these problems while working on projects whose ultimate goal was that of advancing research in the field of autonomous dynamic systems, the scope of this work is to provide a solution that is *reusable*, *modular*, and *maintainable*, and that can be used as a foundation for the development of innovative prototypes and products. With this in mind, the following requirements are formally identified:

(M) Modularity The architecture must constitute both a development and deployment framework for solutions ranging from single hardware or software modules, meant to be easily integrated and reused, to entire autonomous systems prototypes. To guarantee this, the structure of the framework must suit both these use cases: single modules must carry with them all their dependencies,

and it must be possible to easily *merge* single modules to build a "superproject" out of many "subprojects".

(D) Development The architecture must support the development of single modules both as standalone packages, and when part of the software suite of a prototype, *i.e.*, while working on a superproject that integrates a module, it must be possible to modify and update the module, easily propagating the updates to every other superproject which integrates that module, unless such superproject has been specifically configured to use a different version of the module.

(H) Hardware The architecture must offer a *hardware abstraction layer*, *i.e.*, it must make it possible to carry on all the development activities without having to care too much about the underlying hardware on which a module or prototype software suite will be deployed, apart from eventual specific dependencies, and it must ease deployment by offering a common framework across multiple different hardware platforms. In the end, the architecture shall offer very similar, if not equal, development and execution environments across different hardware platforms, requiring minimal effort to be replicated.

(O) Overhead The architecture must be as lightweight as possible, in terms of both computational and storage resources. This is to ensure that the overhead introduced by the framework itself is minimal, and that the resources of the hardware platforms on which the framework is deployed can be used to their full potential for the tasks that the system is meant to perform.

Note that these requirements, if examined by the perspective of subjects such as systems engineering, might not appear well-posed due to, *e.g.*, the lack of a clear and objective way to evaluate them. This is intended though, since they are meant to guide the design of an architecture and not that of a specific system. Moreover, the

design stage at which they are introduced is one in which many things still have to be evaluated such as, *e.g.*, hardware platforms to support and preexisting software tools to integrate, and thus the final definition of the architecture will be the result of many trade-offs. They are high-level requirements meant to guide the design process, characterizing the solution that must be developed from a broad and functional point of view.

Coming back to the generic, although representative, example provided at the beginning of this Section, it appears clear how a framework that meets these requirements would be beneficial, cutting down to the minimum all of the overhead that is usually due to integration and basic organization activities in the development of an autonomous system. Such an architecture would, in fact, provide a common base layer that would both allow developers and teams to focus on their project- or research-specific tasks in a common environment, and ensure that all the work done is reusable, maintainable, and easily deployable on a variety of hardware platforms, ideally ranging from the simplest SoC to the most complex general-purpose computer.

2.3 Related work

To further motivate the premises of this work, this Section presents some of the most relevant solutions that have been proposed in recent years to address the issues outlined in Section 2.1. The list, although short, is in fact exhaustive: to the best of the author's knowledge, and at the time of writing, no other such framework has been developed and proposed in the scientific literature. Furthermore, a review of the works presented hereafter would reveal to the interested reader that none of them fully meets the requirements identified in Section 2.2, especially those that deal with the independence from the underlying hardware and the construction of a replicable

development environment. Finally, to discuss the following, it is inevitable to mention solutions and tools which will be the subject of subsequent Chapters: they will be presented in due detail in the following, while proper references are provided here.

2.3.1 RoboStack

The RoboStack framework [2], published in 2022 and maintained ever since, constitutes a first promising solution to the challenges outlined in Section 2.1. RoboStack is a collection of software packages built on the Conda package management system [3], which has become the de-facto standard in both industry and academia for the management of Python *environments*, a concept that has gained particular interest in the scientific community in recent years thanks to the popularity of Python as a programming language in contexts like big data processing, statistics, machine learning, and artificial intelligence. The Conda package manager allows to specify a set of software modules to be collected and installed in an isolated location, together with their specific versions in case such a level of granularity becomes necessary. Once such a configuration is specified, it can be replicated with ease on any other machine and filesystem path, regardless of the operating system and many other factors, thus providing a way to ensure that the software environment in which a program is developed and run is always the same. RoboStack builds on this concept, providing a wide set of software packages and libraries that are commonly used in the development of robotic systems, such as the Robot Operating System (ROS) and the Gazebo simulator, illustrated in Section 3.1.3, and that are sometimes difficult to install and configure correctly, especially on embedded platforms. The framework is designed to be used in conjunction with the Conda package manager, thus supporting a variety of operating systems ranging from Linux to Windows and macOS, and it is meant to be installed on machines that will be used for the development of robotic

systems, deploying environments through the selection of specific sets of packages to install in each. This solution, although promising and already widely applicable, does not completely meet the requirements previously identified, for three reasons. First, it is entirely dependent on Conda which, although a very powerful tool, may not be the best choice in terms of *isolation* of the software environment: while Conda allows one to install software packages in an isolated location, it does not provide a way to define a specific portion of system resources to offer to that software, a feature nowadays informally called *containerization* and explained later on, which could be useful to further tune the performance of the software running on the system and ensure that the host OS installation stays intact. Second, every piece of software must be added to RoboStack to be installed in one of its environments, which means that the framework is not as flexible as it could be, as that it is not possible to install any software package available from any source out of the box; the addition process requires a variable amount of work, particularly due to the many dependencies a software module may have, as explained in [2]. Last, RoboStack is dependent on Jupyter for development, which is not the best choice in terms of development environments. While Jupyter is a very powerful tool, its main purpose remains that for which it has been originally developed: offering a way to write and run Python code multiple times in interactive notebooks displayed in a web browser, which is not really adherent to the scenario outlined at the beginning of this Chapter which involves developing and testing in real time the software that will run on an autonomous system. Moreover, many plugins existing for other visualizers and IDEs, some of which are presented in the following Sections, have to be reimplemented to be used in a Jupyter environment, which is not always possible or convenient. These limitations, although not critical, make RoboStack not the best choice for the development of autonomous systems, especially when the requirements identified in Section 2.2 are taken into account.

2.3.2 eProsima Vulcanexus

The Vulcanexus software suite [4] developed by eProsima, originally released in early 2023, poses itself as a simple framework for the development of autonomous robotic systems. The suite is composed of a set of software packages that are meant to be installed either on a host operating system or inside a *container* managed by, *e.g.*, the Docker Engine presented in Section 3.3.1. The suite is fairly lightweight but nonetheless comprehensive: it provides the Robot Operating System 2 (ROS 2) middleware, together with the Fast DDS by eProsima implementation of the Data Distribution Service (DDS) communication middleware introduced in Section 3.1.1, plus some more software tools aimed at more specific scenarios like, *e.g.*, that of microcontrollers. Moreover, the fact that it is shipped also as a Docker image, compatible with different hardware platforms, adds some flexibility to it in terms of isolation of both the development and execution environments with respect to, *e.g.*, the aforementioned RoboStack framework. However, the major limitation of this framework lies in the fact that its features more or less end here. With respect to the requirements listed in Section 2.2, no effort is made towards the construction of a modular development and deployment environment, nor towards the support to a wide range of hardware platforms, especially in the case of SoCs aimed at robotic systems which, as will be explained later on, are gaining an increasing interest by the scientific community but require a fair amount of effort to be correctly set up. Furthermore, the restriction to the Fast DDS communication middleware for ROS 2, although evidently motivated by the fact that it is also developed by eProsima, hinders the flexibility of the framework in terms of communication protocols, an aspect that becomes critical in some operational scenarios where, *e.g.*, the network is highly subject to packet loss as explained in Section 3.1.2.

2.3.3 NASA F Prime

The F Prime framework [5], developed in the United States by the Jet Propulsion Laboratory (JPL) of the National Aeronautics and Space Administration (NASA) and published in 2018, is currently one of the solutions that better fit the requirements listed in Section 2.2. It is a lightweight, modular, and open-source framework for the development and deployment of software for small atmospheric and orbital vehicles like the CubeSats and the Mars Helicopter prototype. It provides the foundations for a software architecture oriented at flight control in which software modules, written in the C++ and Python programming languages, can be loaded and configured easily as *components* of the architecture itself. The framework, due to its organization, encourages the development of software packages meant to be published and reused among subsequent missions and prototype developments. Moreover, the framework offers ad-hoc communication primitives and services, as well as some thread scheduling capabilities, to ensure that software tasks in the architecture meet their real-time requirements, an aspect that is crucial in flight and space operations. Finally, thanks to its light organization and basic system requirements, it supports a wide variety of general-purpose and real-time operating systems (RTOS) and can easily be compiled on many hardware architectures. This framework is very promising and shows a lot of potential, but still does not fully meet the aforementioned requirements, mainly because of its limited diffusion and specialized scope. Although it is open-source, it is aimed at a very specific scientific sector, that of space missions, thus receiving limited acclaim outside of such area. Moreover, a downside of its lightweight nature is that it offers limited support for the integration of third-party modules and libraries, a fact which limits the capability of the framework of being used in a wider range of scenarios comprising, *e.g.*, that of swarm aerial robotics, as well as the potential benefit of contributions from the scientific community. Finally, as declared in [5],

in its current version it depends on commercial, licensed software for some of its functionalities, which could be a limitation for users outside NASA and JPL.

2.3.4 MARTe2

The MARTe2 framework, standing for *Multi-threaded Application Real-Time executor 2* [6], is the second qualified framework that is presented in this Section. Originally developed in 2005 by a team of nuclear fusion researchers at the Joint European Torus (JET) UK facility and then registered as a work of the Fusion For Energy (F4E) european organization, it has been under active development ever since and is currently employed in many nuclear fusion research facilities around the world. As its name suggests, it is a framework aimed at the development of real-time software control architectures. It is written in C++ and supports only this programming language. Its organization is actually the closest to what is requested in Section 2.2 and thus deserves a little deeper analysis. It is composed by a core set of libraries plus a set of optional "components" that add specific functionalities or support for particular hardware devices, algorithms, communication protocols, etc. As illustrated in Figure 2.1, the core libraries are divided in three distinct groups:

- `BareMetal`: a set of libraries that implement basic functionalities ranging from data types and hardware abstraction, to communication primitives and logging, and provide the definition of base classes that can be used to develop new components of an architecture.
- `FileSystem`: these libraries provide functionalities to read and write data from and to files, as well as to manage the configuration of the system and the logging of the data produced by the components.

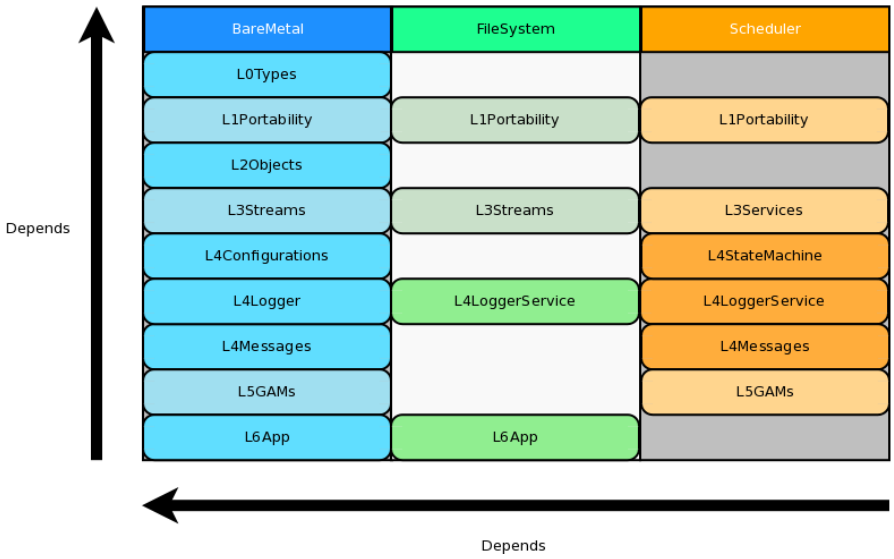

Figure 2.1: Organization of the core libraries of the MARTe2 framework.

- `Scheduler`: libraries in this group implement real-time thread schedulers and their algorithms, as well as models such as state machines that can be useful to keep track of the state of a software component or algorithm.

Moreover, note that libraries in each group follow a hierarchical organization, with those on the lower levels providing basic functionalities and those on the higher levels providing more complex and specialized ones. All of the framework and its components can be built using the GNU[IV] `make` tool, as every MARTe2 software package comes with its own `Makefile` and the entire framework relies on a system of Makefiles that define environment variables and search paths for the compiler, then finally invoke it. The vision guiding the structure of the framework, although it has now reached a maturity that could make it applicable also in contexts that are completely different from nuclear fusion, *e.g.* again, that of robotics, is still that of its early days: implement real-time software control architectures for physics research plants. Thus, its scope has remained the same ever since: managing software units

[IV] GNU is Not Unix, https://www.gnu.org/.

that run *algorithms* and exchange *data* organized in *signals*, which are collections of data of a specific type and dimension, modeling algebraic objects ranging from scalars to vectors and matrices. This idea reflects the main classes of software components that can be found and implemented in any MARTe2-based architecture, and that are the following:

- `DataSource`: a component that exchanges data organized in signals with the external world, *e.g.*, a hardware device or another software component.
- `Broker`: a component that manages the exchange of signals between higher-level components and `DataSources`, implementing memory synchronization schemes if required.
- `GAM`: short for *Generic Application Module*, a component that implements an algorithm that processes input signals and produces output ones, *e.g.*, a control algorithm or a data filtering algorithm.

Finally, a MARTe2-based architecture is usually divided in one or more applications, intended as processes running on the host machine, which contain a number of components specified in a configuration file together with their configuration parameters. Such configuration files, informally named `cfgs`, have a specific syntax, allowing an application to be built by literally "assembling" portions of a configuration file that define the components and their parameters, and then running the application with a specific configuration file that specifies the components to be loaded at run time, plus the settings for the OS scheduler. The reason why this work exists and MARTe2, although being the most promising solution, is still found in this Section, is that as every other work hereby discussed it has its own downsides that set it a bit too much apart from the requirements stated earlier in this Chapter. The first ones descend from

its original scope, and recall the ones commonly found in previously examined works: it has been developed to aid in a very specific scientific research field, that of nuclear fusion in this case, and thus found very little to no adoption in the years outside of it; this translates to limited improvements over the original organization, and lack of support for modern technologies, third-party libraries, and use cases. The code organization and build system described early on is functional and efficient but follows old practices, like the focus on signals rather than software components, and relies on tools like `make` which, although standard and widely adopted, have nowadays been superseded by more modern solutions which are more suited to the management of large and modular codebases. Due to its strict real-time nature, MARTe2 also imposes some restrictions on the development of software modules, which must be written in C++ and must adhere to old language standards, lacking modern features and erecting a barrier against the integration of many open-source libraries and tools. The direct experience of the author, which worked with this framework in both research and didactic settings, suggests that these facts, together with the effective but cryptic syntax of the `cfgs`, make MARTe2 a very powerful but also clunky framework which often scares away newcomers and makes it difficult to maintain and evolve the software architectures built with it. Finally, although its compilation requires very little features from the host system, it is not officially supported on architectures other than `x86` and requires a tailored OS kernel to fully exploit its real-time capabilities, limiting its application in practice to Linux-based systems if one wants the smoothest possible usage experience.

3 Tools for the job

The previous Chapters highlighted the need for a new software architecture that may constitute the foundational layer of distributed autonomous systems, and provided a solid set of requirements that can drive its design. Existing frameworks have been reviewed, some of which provided valuable insights and ideas about how to address some of the design challenges, but none ultimately offering the complete set of features required. Before presenting the design of the solution that this work proposes, it is important to delve further into a point that the previous discussion shed light onto: there are solutions available in the literature, in the industry, and in the open-source software community, that can be employed to address some of the key issues that the requirements identify. Following the computer science principle recalled in Section 2.2, that is, to not reinvent the wheel, it would be wise to review these tools and see how they can be integrated in the proposed solution. This Chapter provides such an overview: for each tool, its necessity will first be motivated by illustrating a problem that is common in the development of distributed autonomous systems, and then the tool itself will be presented, along with a brief discussion of its features and limitations. Note that the aim of this work is to provide a general-purpose software architecture, so not all of the solutions presented in this Chapter, as well as all the features that the proposed architecture will have, may be necessary or even beneficial in every practical context. However, the goal is to provide a solution that can be adapted to a wide range

of applications, as a good toolbox to have at hand, offering a set of features that can be configured and used as needed. Indeed, recalling that one of the requirements is also to develop an architecture that can be maintained over time, the intention is that the tools presented in this Chapter will be included because they represent valid solutions at the time of writing to the problems they address, which are relevant in this context, but since not a thing of this is set in stone each aspect of the proposed architecture can be subject to change in the future should the need arise. To conclude the Chapter, in a similar fashion, a set of hardware platforms of interest are presented, which can be used to implement the proposed architecture in the real world and are as well the subject of further developments.

3.1 Inter-process communication: the middleware

When developing distributed systems composed of many modules, especially software ones, the transmission of data among them becomes a topic of paramount importance. Note that this issue concerns both transmissions happening between software modules running on the same host and on different hosts. Moreover, alongside this topic there is that of *intra-process communication*, that is, the organization and regulation of data exchanges among different threads running in a same process. Examples of situations in which many software modules may coexist inside a single process, and why such a situation may be desirable, are presented and discussed in the following, for now let us focus on the general problem of inter-process communication and simply note that in the majority of the situations, this other problem can be solved by employing either the same tools that are used for inter-process communication, or simpler ones based on shared memory areas guarded by synchronization primitives. One has, in general, to figure out a variety of issues such as the following:

- **Data formatting**, *i.e.*, how information that should travel from one module to another should be encoded and structured in packets or similar entities.
- **Data serialization and deserialization**, *i.e.*, how packets should be transformed into a format that can be transmitted over a medium, which is usually some kind of point-to-point connection or a network.
- **Data transmission**, *i.e.*, how packets should be sent and received, and how the system should react to errors or delays in the transmission; this also includes things like reliability requirements, that is, whether the system should guarantee that a packet is delivered, and how many times it should try to deliver it if it fails.

The points listed above are usually dealt with by employing a specific *protocol*, that is, a predefined set of rules that regulate all those aspects of the communication. All the software that composes the system should adopt such protocol, which is usually implemented in a set of libraries that can be used in the code. However, dealing directly with networking protocols might slow down the development process: the host system configuration usually becomes very important, since all the operating systems involved must agree on the protocol implementation to support and use, and a lot of work may become necessary to proficiently use all the protocol APIs to achieve all the desired communication features and behaviors. Let us remember that this is not the original task: the goal is to develop a distributed autonomous system, and to write software for it, thus it would be desirable to have a tool that abstracts all the networking details and provides a simple and easy-to-use API that makes data transmission in software modules a straightforward operation, from both the coding and implementation points of view. This is where a particular kind of software, historically denoted by the term *middleware*, comes into play. To grasp its importance in the context of this work, a step back in time is necessary.

In 1968, a conference sponsored by the Science Committee of the North Atlantic Treaty Organization (NATO) was held in Garmisch, Germany, on the topic of software development. Although not widely discussed, it was a historically significant event in this field: it provided a comprehensive overview of the design practices and methodologies used by leading companies in the growing and highly competitive field of computing. The goal was to identify standard strategies for addressing common problems. It should be noted that at that time, and for at least another fifteen years, there were virtually no standards in computer production and programming. Each company offered its own product developed from scratch, in both hardware and software. Standardization and collaboration among various manufacturers began only when information started being exchanged between different computers, first through storage media and later over the Internet, alongside a growing demand for new types of devices and standard formats to adopt when sharing data. A fairly complete transcription of the conference can be found in [7]. It includes the first official mentions of several concepts well known today, such as *component design* and *unit testing*. However, of particular relevance to this work is the representation of the hierarchical structure of software presented by one of the speakers. This structure, valid even today with certain extensions, is illustrated in the inverted pyramid scheme shown in Figure 3.1. The idea is that a complex system is like an inverted pyramid: it cannot remain operational without adequate support provided by secondary components. These might be completely invisible to the end user and thus may be considered somewhat external, yet are of vital importance for the correct functioning of the whole: *compilers* and *assemblers*, for instance, which are programs that translate code written in a given language first into the target architecture's assembly language and then into binary machine language, typically generating an executable file at the end of the processing pipeline. These tools must be reliable and efficient, as the imple-

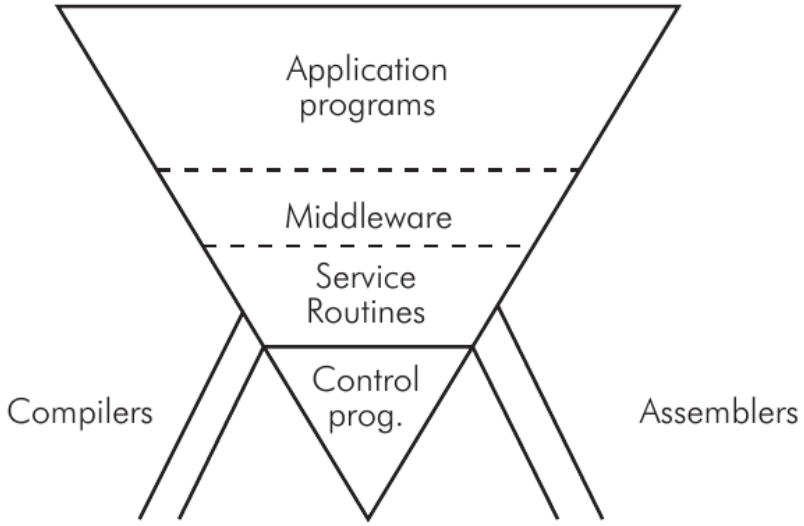

Figure 3.1: Software organization in a general-purpose computer, as presented at the 1968 NATO conference on software engineering.

mentation of everything else depends on them. Next, we have the layers of *Control Programs* and *Service Routines*: these are what we today call the *operating system* and *device drivers*. They are typically developed with a strong dependence on the machine at hand, considering its specific characteristics, and heavily depend on the employed compilation tools. Their role is to provide an abstraction of the hardware that makes its use simple both for the end user, who interacts with applications, and for application developers themselves. The application software is precisely what we find in the topmost layer: programs that use the hardware more or less directly to provide various services to users. There is, however, an intermediate layer that is often overlooked because it was not always present or was difficult to characterize in the past: the *middleware*. Situated between system software and application software, middleware found its first definition and characterization at the aforementioned conference. Initially conceived merely as a connective layer between current and *legacy* software, middleware provides services and functionalities common to applications beyond what the operating system offers, building itself on top of it and abstracting

its own implementation and protocol details. From the perspective offered by the topmost level of the inverted pyramid it appears as normal application software, often consisting of shared or dynamic libraries. Middleware typically offers services to programmers aimed at, *e.g.*, data management, communication, and authentication. Therefore, middleware aids developers in building applications more efficiently by acting as a connective tissue between applications, data, and users. In distributed environments, middleware can facilitate the development and execution of large-scale applications. A typical service offered by middleware is a method for data exchange between applications, even across different devices, according to a common format referred to as an *interface*, which is defined as needed. In general, the services offered by middleware are specified only in terms of their dynamics but can be implemented on different architectures and devices, functioning consistently while keeping their implementation hidden from both the application programmer and the end user. For example, when middleware handles communication between applications, it might position itself within the network stack between the transport and application layers[I]. Today, middleware can be categorized into two types: that which operates in *human time*, designed for direct use by human users (*e.g.*, web services), and that which operates in *machine time*, composed mainly of APIs that application developers can use to facilitate their work, benefiting from increased portability across a wide variety of devices. In light of all this, and considering the common difficulties faced when designing the software for a distributed autonomous system, it becomes clear how adopting communication middleware to solve the problem of data exchange among modules can simplify things: many functionalities that would have to be implemented from scratch would instead be offered by the middleware itself as services to be employed in the programs being developed. The selection of this particular class

[I]Considering the TCP/IP model, which is a four-layers reduction of the original seven-layers Open Systems Interconnection (OSI) model of the networking protocol stack.

of middleware is again motivated by both the current trends in the software industry pertaining to the development of autonomous systems, and to the author's experience in the field during the years that led to this work.

The rest of this section will introduce three specific middlewares that are widely employed in the industry and in the open-source community, and that have become of pivotal importance in the two practical scenarios considered later in this work as a specialization of the proposed architecture: that of autonomous robotics and that of power plants. The first middleware is the *Data Distribution Service* (DDS), which has been the first to provide a standard for data exchange in distributed systems, followed by the Zenoh middleware which addresses some of the limitations of DDS, and finally the *Robot Operating System* (ROS), which is a middleware specifically designed for robotics applications. All these three components will be included in the architecture proposed in the following Chapters, making some of its foundational layers.

3.1.1 Data Distribution Service

In 1989, eleven companies including Hewlett-Packard, IBM, Sun Microsystems, Apple Computer, and American Airlines, founded the *Object Management Group* (OMG), a consortium that is still active in the field of computing with the goal of defining standards for industrial software. This effort aims to foster the integration of solutions from different vendors into unified ecosystems, covering a variety of applications and technologies. The purpose of the OMG is to establish, for every new type of software under consideration, a common model to be followed during its implementation, thus enabling and promoting its development and use on any potentially relevant platform. The group provides the industry with specifications only, not implementations. However, any team submitting a new specification must provide a potential

implementation before it is accepted, intuitively to avoid defining impractical or useless standards. OMG member companies encourage the development of solutions that can interoperate immediately without any limitations.

One of the standards defined by the OMG is the *Data Distribution Service* (DDS) [8, 9]. It specifies a particular type of middleware responsible for managing direct communications between computer systems, including the details of how data should be serialized, deserialized, transmitted, and routed, as well as how APIs should be structured for programmers to invoke these operations. Its definition has been carried over with the goal of allowing implementations to satisfy real-time determinism guarantees whenever necessary. Moreover, the goal of the DDS is to enable technologies, platforms, and devices from different vendors, operating in distinct contexts and locations, to immediately communicate, exchanging data for which only the format and minimal other attributes must be known beforehand. This facilitates the management of critical aspects, like performance and scalability of the various components and the network connecting them. Effectively, the DDS greatly simplifies what would otherwise be for the programmer a complex task of configuring networks and organizing data transmissions, which, as discussed, is one of the classic challenges faced during the development of a distributed autonomous system. As such, the DDS has become widely employed in industrial contexts ranging from manufacturing [10] to power grids [11].

The middleware model follows a *publish-subscribe* paradigm: entities involved in the communication declare their intent to transmit or receive packets of information, which are encoded according to appropriate interfaces, being named informally *publishers* or *subscribers* and instantiated as `DataWriters` or `DataReaders` respectively. The publish-subscribe denomination arises from the way different transmissions

are organized, from the programmer's perspective, into channels denoted as *topics*. Topics are defined by:

- a **name** that identifies the topic, generally encoded as a text string to be easy for programmers to remember and use;
- an **interface**, described using a specific language (*e.g.*, an Interface Description Language (IDL)[II]), which specifies the content of a single data packet and how it is encoded.

Interfaces are intended as structured data types, *i.e.*, data structures containing ordered fields of different types, ranging from simple integers and floating-point numbers to more complex structures like arrays and strings, or other interface types as well. One key aspect to consider is that, given an interface specification, the middleware will take care of generating the code necessary to serialize and deserialize data packets according to that interface, as well as to transmit and receive them over the network. This is a significant advantage, as it allows the programmer to focus on the data itself and on the logic of the application, without having to worry about the details of the network communication. Another key aspect to consider, which also makes the DDS very versatile, is that communications are asynchronous by nature: `DataWriters` publish data on a topic regardless of the presence of any `DataReader` listening on that topic. `DataReaders`, on the other hand, can subscribe at any time to a given topic and start receiving messages from that point on. The OMG DDS standard provides APIs for both synchronously waiting for new messages, as well as asynchronous notification and handling via *callback*[III] functions.

[II] The acronym IDL identifies a class of languages that allow the description of operational elements intelligible by different languages. Various IDLs exist, and in this context, DDS employs one to make the specification of a data packet format simple for programmers, although no specific IDL is mandated by the OMG standard.

[III] A callback, in a software program, is a routine that gets called asynchronously, *i.e.*, when an event

The behavior of any entity in a DDS network, and thus that of every transmission, can be configured by specifying a Quality of Service (QoS) policy for it [12]. A DDS QoS policy typically consists of items such as:

- the **history policy**, which indicates whether all packets transmitted on a specific topic should be delivered or buffers, limited to a certain size, should be put in place and rotated in case of network congestion;
- the **history depth**, which is the depth of the finite queues optionally set in the previous point;
- the **reliability policy**, indicating whether communications should be managed by the middleware to ensure that all subscribers of a topic receive all packets transmitted by publishers, or packet drop and loss are allowed, thus enabling *retransmissions* if necessary;
- the **durability policy**, allowing transmitted packets to be retained by publishers and delivered to subscribers that may connect later;
- the **deadline**, **lifespan**, **liveliness**, and **lease duration**, which are characteristic maximum times to determine when a packet has become outdated or an entity can be considered inactive.

Of course, there are default settings that are kept as consistent as possible across different implementations to ensure maximum compatibility. A complete scheme of a DDS network of applications can be found in Figure 3.2.

Even at this level, the significant simplification introduced by the DDS middleware becomes evident: with a relatively simple and quick configuration, issues such as

occurs for which that callback is registered, and either an event handler or a job scheduler invoke such callback to process the event.

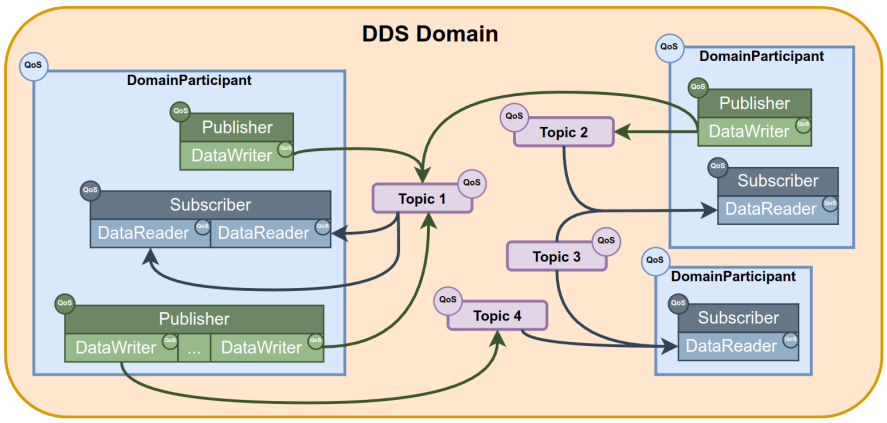

Figure 3.2: A network of applications using the DDS, each denoted by its own `DomainParticipant` [13].

choosing a transport protocol (*e.g.*, Transmission Control Protocol (TCP) or User Datagram Protocol (UDP)) and developing software modules to use it for transmitting information encoded according to specific rules and formats can be immediately resolved. The DDS takes care of all tasks necessary to organize communications: from the most basic, such as message transmission, to more organizational tasks like publisher advertising and node discovery within the network, as well as flow control and retransmissions. It also handles publisher redundancy and other pathological situations—all in a way that is completely transparent to the programmer. DDS, in effect, makes it very simple to build distributed applications or networks of applications, as it hides and simplifies many management tasks, allowing the developer to be abstracted from factors that could otherwise prove complex to manage. Commercially available DDS implementations today offer API libraries in various languages, such as Ada, C, C++, C#, Java, Python, Scala, Lua, Pharo, and Ruby, and since they are all built on the same standard, they are fully compatible and interoperable. There are also specifications concerning lower-level types of DDS, designed to be included in resource-constrained systems, such as embedded systems, while still maintaining compatibility with more comprehensive high-level implementations.

Communication modes, serialization, and deserialization of data are defined in the OMG standard called *Wire Protocol*. Despite the broad scope of this specification, it is instrumental to highlight two key parts for the benefit of the following discussions.

Discovery Module

This module of the DDS includes protocols that allow different DDS instances running on network-connected systems to discover each other and organize potential communications from that point on. Generally, there are two such protocols:

- **Participant Discovery Protocol (PDP)**, which allows DDS instances—each associated with a particular application running on a specific machine, hence called *participants*—to announce their presence to others while simultaneously obtaining information about any other reachable participants.
- **Endpoint Discovery Protocol (EDP)**, which comes into play immediately after PDP, and relates to transmitting information about the *endpoints* owned by the various participants. These endpoints, as publishers or subscribers, are deemed compatible if they share the same topic and interface, and if the QoS policies *offered* by publisher entities satisfy the *requests* of subscriber entities.

Implementations can choose to include, beyond the PDP and EDP protocols defined by the standard[IV], proprietary versions. If two instances initiating communication have at least one PDP and one EDP in common, they can exchange information and complete the discovery process.

These protocols would be located at the Internet Protocol (IP) level in a network stack, and can be implemented in either *unicast* or *multicast*, depending on whether all

[IV] These are known as **SPDP** and **SEDP**, respectively, where "S" stands for "Simple".

participants are known in advance or not. In the former case, preliminary network configuration is required to preset the addresses and port ranges of all involved machines, while in the latter, the protocols will automatically manage everything as soon as the DDS instances are launched. The potentially vast operational scale of these solutions has made DDSs attractive for mission-critical areas such as military and logistics, although, for now, relying entirely on the Internet for such communications is not advisable. Unless predefined configurations are in place, it is necessary for every network device along the path between two participants to support multicast and not block such transmissions.

Platform Specific Model

This model establishes how data of different formats and encodings should be serialized, transmitted, and deserialized in a uniform manner across all possible implementations and platforms. In an IP network, all data packets are transported over UDP. Whether the endpoints are associated using multicast or unicast, the Discovery Module's protocol opens UDP *sockets* for them with well-defined port numbers, which depend on:

- the **domain ID**, a numerical constant that selects a range of UDP ports in the implementation, allowing applications using DDS to communicate simply by specifying the same domains; multiple domains can coexist even on a single machine;
- the **base range** of UDP ports available in the system;
- the **participant ID**, a unique identifier for an instance within a machine;
- any other **manual**, specific configurations.

One may wonder why UDP is used for data transport. The reasons are similar to those in other contexts where a deterministic behavior is desirable: UDP is a basic, best-effort protocol without any communication management features beyond simple transport between endpoints. As such, it is lightweight and supported by a wide variety of devices and platforms. This simplicity gives UDP implementations complete predictability, excluding the unpredictability of the network itself, and natural scalability. Therefore, it is the best choice for real-time systems as well as for the development of this type of software.

3.1.2 Zenoh

The Data Distribution Service middleware effectively solves the inter-process communication problem in distributed contexts, allowing applications to easily set up communication channels over a network of potentially very large scale. As the previous Section clarified, it is particularly user-friendly from the point of view of the application programmer also because of its automatic configuration capabilities: thanks to the Discovery Protocol, DDS participants and their related entities can automatically discover each other and connect, without any prior network configuration required. If all of this seems too good to be true, that is because it actually is. Coming back to the origins of the DDS specification, one can see how they date fairly back in time: the first drafts of the protocols were redacted in the late 90s, while the first officially published implementations are from the early 2000s. In the meantime, the world of computing has changed a lot, and situations that were only theorized before are currently the norm. A wide variety of new technologies, which are all somehow related to networking, have seen the light of day in the meantime. As an example, consider the advent and diffusion of various high-bandwidth wireless networks like WiFi or 5G, which then led to other advancements like cloud computing, Internet

of Things (IoT) and edge computing. All these technologies are nowadays part of the daily life of many people, and they have all contributed to shape the current scenario in information technology: digital devices are everywhere, on every scale, all interconnected at the same time using a variety of technologies and protocols. That is also what made distributed autonomous systems possible, but also changed the world into something much different than the one in which the DDS was born. When the OMG DDS standard was designed, networks were still made of cables and routers, with point-to-point links which finally allowed little to no packet loss, and connected devices which were different declinations of the same concept: a big computer. Nowadays, the term "computer" has a much broader meaning, and networks connect devices that operate on many different scales using multiple types of physical links. One of the few things that is common among all is that they all need to share large, if not huge, amounts of data among themselves. This is where the DDS starts to show its limitations: it was designed for a world where the network was a mostly reliable and predictable medium, and connected devices could be assumed to have a certain level of computational power and memory. This is not the case anymore, and the DDS is starting to show its age. The main limitations of the DDS are related to these exact contexts, and came up in the last five years due to its wide adoption in new contexts like that of autonomous robotics, where having multiple heterogeneous devices connected over a wireless, lossy[V] network is the norm. The most limiting factor of the DDS is, in fact, its intrinsic reliance on a network layer which requires little to no retransmissions. If this assumption is not met, communications based on the DDS can become very slow or even fail completely due to network congestion. This is mainly due to the fact that the Discovery Protocol has not been designed with

[V] In this context, the adjective "lossy" denotes a packet switching network subject to *packet loss*, *i.e.*, the possibility that packets flying get dropped by the devices handling them because of network congestion or similar phenomena, in an attempt to preserve overall network functionality.

this situation in mind, so that it requires a number of multicast transmissions that grows exponentially with the number of participants in the network. When the network is not reliable, *e.g.*, in the case of WiFi, these transmissions can be lost leading to bursts of retransmissions that might clog the network altogether. A second important limitation of the DDS is its inability to optimize bandwidth usage and retransmissions when dealing with large data, *e.g.*, video streams, which obviously were not a concern back in the days in which the DDS standard was defined; this leads to even more risks of ending up with a congested, unusable network. Moreover, these problems become unsolvable if one attempts to route DDS traffic over a Wide Area Network (WAN), such as the Internet, whose routers and switching devices might apply traffic control policies that might throttle or even prevent DDS packets to travel safely; again, this scenario was not considered in the original specification, but has become of pivotal interest due to relatively new concepts such as IoT. Finally, let us note that all of the features and functionalities that the DDS offers come at the price of an overhead in terms of both resource usage and protocol complexity. A straightforward DDS implementation that has all the features specified in the standard typically ends up being quite a large piece of software, not so easy to adapt to resource-constrained environments like embedded systems, and a DDS data packet usually has many control fields that increase the amount of data being transmitted.

This discussion highlights how the DDS middleware is useful in some contexts, mainly those in which the network is fully reliable and ease of configuration is of the essence, but inapplicable in others. This is where a new communication middleware, recently born as a successor of the DDS to address the challenges of the modern, interconnected world, comes into play. This middleware is called *Zenoh* [14], which is the crasis of “zero network overhead”.

The design of the Zenoh protocol starts from the following consideration: an application programmer needs communication APIs to handle three main types of data, which are:

- **data in motion**, *i.e.*, data that is being transmitted over the network among multiple devices and software modules, all active at the same time;
- **data at rest**, *i.e.*, data that is stored in a device and can be retrieved at any time by other devices;
- **data for computations**, *i.e.*, data that is used as input for computations requested from one network endpoint to some other(s) and that is produced and returned as their output.

The publish-subscribe pattern was invented to handle data in motion, which eventually led to the definition of the DDS among many other protocols and to the diffusion of distributed systems, and is still the best solution to apply in this case. The last two sets of data, from a communication perspective, can be handled in the same way by implementing an API that allows to *query* network endpoints, formalizing both the interrogation phase and the answer phase. Zenoh is defined as a Pub/Sub/-Query protocol, since it implements three fundamental kinds of objects: publishers, subscribers, and queryables. Each object, once instanced, can *declare* itself as one of the three kinds, then exchange data according to the operations permitted by its type. This is very important because in many contexts in which the DDS is applied today, *e.g.*, that of autonomous robotics, there is in fact the need for a query-based or client-server type of communication which is not defined in the protocol, thus leading to its re-implementation on top of it in any context and requiring at least two DDS topics and a control protocol to organize the transmissions; this contributes to

the overheads and the issues mentioned above.

In Zenoh, each data point is organized as a pair of type *(key, value)*, where the value is the actual data and the key identifies the communication channel, or channels, which that value concerns. A key is typically an array of characters made of many substrings separated by the / delimiter, as in `home/kitchen/thermostat`; wildcards and regular expressions are also allowed in API calls to match multiple keys at once. Note how the use of the delimiter naturally enforces a partition of the space of keys, and thus of the data flowing in the network, into multiple subdomains. This is a key feature of Zenoh as it allows to optimize many phases in which this topology is involved like, *e.g.*, the discovery of network endpoints: as opposed to the DDS in which all endpoints must know everything about each other to be able to exchange data, in Zenoh endpoints query for their fellows only when they must share data that has a specific key, and discovery-related transmissions are directed only to hosts and endpoints that handle that key, starting from its left-hand side going towards the right-hand one. This implies that when, *e.g.*, a new subscriber wants to receive data published with the `home/kitchen/thermostat` key, it will first query which endpoints handle the `home` subdomain, then ask them directly for endpoints handling the `kitchen` subdomain, and so on until the end of the key. This way, the number of transmissions required to discover and connect endpoints is effectively reduced, since these transmissions are triggered only when necessary and happen among only the hosts and applications that are actually involved. To achieve this huge optimization over the DDS, however, Zenoh abandons the automatic discovery feature: there is no such automatic discovery functionality in Zenoh, and all applications or hosts must be configured manually to know which other such entities are present in the network and how to reach them. Multiple network topologies are possible, as explained in [14] and illustrated in Figure 3.3:

- **clique**, where all endpoints are peers and can communicate with each other;
- **brokered**, where communications are routed among endpoints by a central router application, which is implemented as a daemon[VI];
- **routed**, in which many routers enable communication among multiple groups of hosts and applications in brokered mode;
- **mesh**, a hybrid situation in which endpoints may share partial network structure information or even route data among themselves.

This may seem a step back from the DDS specification but if one recalls that the Discovery Protocol is among the first causes of trouble when the DDS is used in relevant contexts, such as that of autonomous robotics, this has rightfully been deemed a reasonable price to pay to have a reliable and efficient communication.

Finally, another positive aspect of Zenoh is that it is implemented with the additional goal in mind of having the lowest possible resource and memory footprint, which implies that it can run on a variety of different hardware platforms ranging from servers down to microcontrollers. For example, the control fields in its data packets can take up to as little as 5 bytes, as opposed to the at least 56 bytes required by the DDS. Moreover, it is almost completely agnostic of the underlying network layer, positioning itself in the protocol stack eventually as low as on top of the data link layer as showed in Figure 3.4, thus supporting a wide variety of protocols and physical links including UART, Bluetooth, LoRa, Unix Sockets, TCP/IP, UDP/IP, QUIC, WebSockets, and CANbus, offering the same set of APIs and functionalities in all cases. Thanks to this level of versatility, Zenoh is a very good candidate to be used in the development of distributed autonomous systems, especially in those contexts where

[VI] A daemon is a software that runs in the background, typically started when the system is booted and running continuously until the system is shut down.

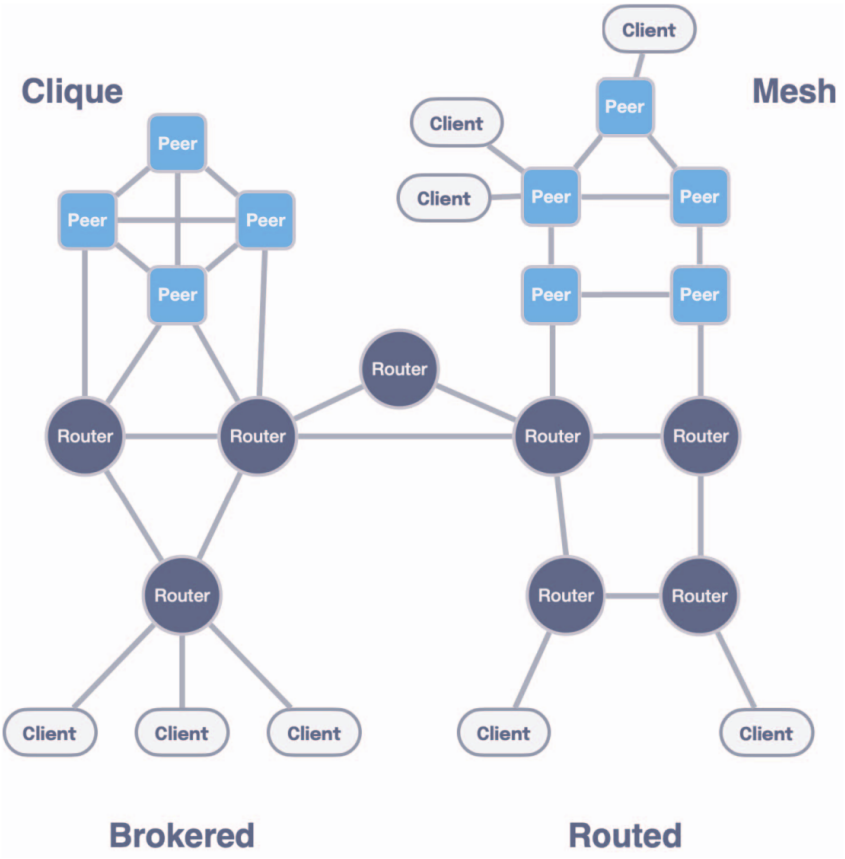

Figure 3.3: Possible network topologies in Zenoh [14].

the DDS is not applicable due to the reasons discussed above, and it has already been successfully employed in industrial contexts achieving good performance in terms of both throughput and latency [15, 16] despite being relatively new, as its version 1.0 has been released in October 2024[VII]. For these reasons, it is included in the proposed architecture in its current state alongside some DDS implementations, to provide a complete set of communication tools that can be used in a variety of contexts, platforms, and applications.

[VII] https://github.com/eclipse-zenoh/zenoh/releases/tag/1.0.0

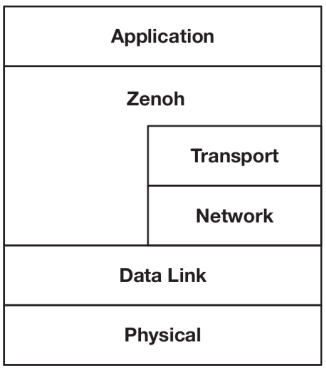

Figure 3.4: Placement of the Zenoh protocol in the network stack [14].

3.1.3 Robot Operating System 2

The previous Sections presented a class of software solutions whose objectives align with those that emerge during the design of a distributed system, and aimed at providing real-time communication services. With all these elements in place, the next solution to be described is a middleware specifically designed for robotics applications, which are a particular case of distributed systems, and that is also widely employed in the industry and in the open-source community. Since autonomous robotics is one of the application settings for which the proposed architecture has been developed and in which it has been tested and employed, this particular middleware is of paramount importance for the rest of this work and will be described in the following in due detail.

ROS 2 [17] is the second version of the *Robot Operating System* middleware, originally developed in 2007 by Willow Garage [18], a spin-off of the Stanford Artificial Intelligence Laboratory. Although its name might give a different idea, ROS is not an operating system in the traditional sense of process management and scheduling; rather, it started as a structured communications layer above the host operating systems of a heterogenous compute cluster. Then, ROS has evolved into a middleware that offers robotic and automation programmers services and APIs such as hardware

abstraction, low-level device control, data exchange and inter-process communication, support for creating client-server modular architectures, and software package management. The entire project consists of a collection of software libraries and tools. It has always been open-source with a community-based development model and rigorous review processes: the source code of each version, ready for compilation and installation, is organized into various repositories hosted on GitHub, and documentation is published online alongside numerous *design documents* that explain and comment on the design choices made and on the new features introduced. Fixes and innovations are tested in a *Rolling* [19] release and subsequently introduced in stable releases, which follow an alphabetical naming scheme similar to that adopted by Canonical for the Linux distribution *Ubuntu* [20] and are thus often referred to as "distributions". Currently, the project is maintained and steered by the Open Source Robotics Foundation [21]. Its open nature led to a steady increase in popularity over the years, and has contributed immensely to a worldwide renewal of the interest in robotics thanks to code sharing and open scientific contributions, both in a theoretical and in a practical sense, favoring the birth of a large number of communities, research projects, frameworks, laboratories and startups all over the world, sparking in the world of robotics a revolution similar to that happened in the software engineering world in the 90s with the advent of open-source software. Today if, *e.g.*, a research team publishes a novel mapping algorithm for 3D LiDAR[VIII] scans, it is very likely that the implementation of that algorithm will be made available as a ROS package, and that it will be tested and reviewed by the community, thus becoming a standard tool for all roboticists to use in their projects.

By its documentation, ROS 2 supports the Linux, Windows, and macOS operating

[VIII] Light Detection And Ranging, a class of sensors that use laser beams to reconstruct the depth of a scene and get a spatial reconstruction of the environment that surrounds them.

Figure 3.5: ROS 2 logo.

systems, but the reality is that support levels are defined by three *tiers* that define support quality and frequency of updates. Currently, the only Tier 1 platform is Ubuntu Linux, in which a ROS 2 distribution can be installed either from binary packages or compiled from source; the latter installation technique can also be applied to different Linux distributions. As for Windows and especially macOS, since their closed nature with respect to Linux usually adds additional difficulties when developing robotics software, they are not so widely adopted and thus the community puts less effort into updating ROS for these platforms, also because of the lack of developers and testers that actually have these platforms available to develop on together with the expertise required to manage and integrate the whole ROS codebase.

For a summary of the history of ROS and an overview of its design goals, see [22]. From that document, it is evident that the project soon proved potentially suitable for complex use cases, such as distributed and real-time management of collaborative swarms of robots or the homogeneous integration of different types of hardware and communication interfaces. A more technical and detailed description of the new features of the version 2, which introduces many of the most popular features, can be found in [23].

Historically, the first service offered by ROS was inter-process communication: in ROS 1, that was implemented by custom protocols built on top of TCP and UDP. The most significant difference between ROS 2 and the first version, and currently its greatest strength, is the fact that instead if reimplementing such protocols it sits directly above a layer, denoted as the RMW or ROSMiddleWare layer, which houses support for various communication middleware implementations aimed at distributed systems, such as the DDS and Zenoh. The first communication middleware to be integrated with ROS was the DDS [24]: the entire middleware was indeed built on top of DDS, whose specific implementation is left to the user and is fully interchangeable, to implement the basic inter-process communication and data exchange services. This means that all the benefits of the DDS mentioned earlier such as ease of use and interface definition, Quality of Service, transmission configuration through discovery, and serialization, are immediately available to robotic system developers, solving quite a few problems but introducing new ones as previously discussed. Developers can also take advantage of ROS's standard libraries and the vast open-source ecosystem of packages based on ROS that are publicly available, as well as contribute their own work to advance the community of engineers and developers who use it.

From its origins, each version of ROS supports the following two programming languages: C++ and Python. This choice has been motivated by the high diffusion of both on nearly every platform, as well as by their intrinsic complementary nature. C++ is the most used and recommended for performance-critical as well as production applications, while Python is more suitable for rapid prototyping and for the development of scripts that interact with the system, as well as for high-level software modules that have no stric performance requirement but whose development and maintenance may benefit from a higher-level language. Moreover, Python is the de-facto standard programming language in contexts like machine learning and artificial intelligence,

both fields that are having a disruptive impact on robotics. The ROS 2 project has also introduced support for other languages, such as Rust, Java, and plain C, and has developed a set of tools to facilitate the integration of these languages with the middleware, thus making it easier to develop software for robots using the language that best suits the task at hand. This is a significant improvement over the first version, which was limited to C++ and Python, and it is a clear sign of the project's commitment to keeping up with the times and the needs of the community.

To provide an idea of the main services offered by this middleware relevant to this work, the key features of ROS 2 will now be described following a top-down approach, eventually reviewing some auxiliary but very useful tools and frameworks; thus, many minor utilities are excluded for the sake of clarity but readers are referred to the technical documentation [25] and basic tutorials [26] for these aspects.

Build system

In the ROS ecosystem as in many similar contexts, software is naturally divided into units called *packages*. The idea is that each package constitutes a standalone software module, designed to fulfill a specific function within the system's overall architecture, potentially interacting with others at various levels and using specific services offered by the system itself through APIs or specialized hardware. Nevertheless, as has been highlighted several times, a robot presents a different scenario from those typical of software development and it is not uncommon for a developer to work on multiple packages simultaneously. Adding to this complexity is the fact that if different modules need to interact, their corresponding packages will inevitably be interdependent. Therefore, a common system is needed to simplify the creation of a *workspace* in which to develop, execute and test the software contained in multiple packages. The

solution provided by ROS 2 consists of a universal build system applicable to any context where this middleware is employed and divided into two modules, with the reasons for this approach described in [27].

Initially, there is *colcon* [28]. It was originally intended to be a tool that automates and simplifies the compilation and installation of Python or C++ code packages via CMake[IX] [29] or Setuptools [30] for use with ROS 2, but soon it became a more general project of its author. Today, it is used even outside of the robotics world to manage large projects and codebases: it is not uncommon to find software libraries and tools whose source code can be managed and built using colcon. ROS 2 relies on colcon to organize every aspect of a workspace, intended as a collection of packages either in source code or binary[X] form. Essentially, colcon creates a workspace by defining some basic directories in the file system where both source code and build artifacts will reside, and then manages compilation and installation according to either automatic or specified rules resolving dependencies automatically, *i.e.*, ensuring that all external pieces of software that are required are actually present on the system and correctly configured. Each package, to be manageable by colcon, must include a *package manifest* in the form of an XML file[XI] in which all the characteristics of a software module, such as its name or dependencies, are listed. Installation may result in the generation of an executable file or a shared library, and possibly also of symbolic links to a location within the workspace where builds are performed; this greatly simplifies

[IX] CMake [29] is a cross-platform free software for build automation, whose name stands for "cross-platform `make`." It was developed to simplify the build process by providing a scripting language to easily automate it entirely even for complex projects.

[X] In this context, the term *binary* identifies a way in which software can be shipped which does not include source code files, and is instead limited to compilation output in the form of either libraries to link against or executables to run.

[XI] eXtensible Markup Language, based on a syntactic mechanism that allows the definition and control of the meaning of elements contained within a document or text. It is used in web pages or for collecting data in a text file so that it is also interpretable by a computer program: parsing can be easily performed using appropriate libraries.

matters when testing the software and compiling it multiple times, as it avoids having to modify the installation points each time, always pointing to the latest executable or configuration file. The many command-line options of colcon also allow interaction, if necessary, with lower-level tools used at various stages of compilation and build.

In addition to the typical challenges related to building interdependent packages, it is also necessary to address those concerning integration with preexisting libraries and interfaces that compose a ROS 2 installation. To address this and many other issues, the second module is used: *ament* [31]. It is a collection of Python scripts invoked automatically by colcon that complete the build process by resolving dependencies with other ROS 2 packages and ensure that paths, environment variables and scripts are correctly set up, so that the software can later be executed without issues and with a minimal set of commands being necessary. This system proves effective: when having multiple packages in a workspace, they can all be built together with a single command. Compare this ease of use and the integration of new solutions like CMake and Setuptools with, *e.g.*, the `make`-only build system of MARTe2 discussed in Section 2.3.4 and you may start to get an idea of the advantages of using ROS 2 in a complex software project for a distributed autonomous system.

Interfaces

In ROS 2, like in the DDS, it is possible to specify the format of the data packets exchanged among software modules, which are called *messages*, using special text files called *interface files* [32]. In these files, similarly to DDS IDL files, the structure of the packet is defined by specifying each field on a separate line by a name and a data type. Data types range from integers or floating-point numbers to strings, and even variable-length arrays. It is considered good practice to group these files into

packages consisting solely of interface definitions, which are then built and installed using the same build system: the result in this case is a collection of Python classes, and C++ header files and shared libraries that define the types of the data packets as well as the serialization and deserialization operations, which can then be included and linked in other modules that need to exchange data in such formats. In practice, a ROS 2 developer never needs to concern themselves with anything related to format specification, serialization, transmission, reception, or deserialization: all of this is automatically managed by ROS 2 via its RMW layer.

Unlike a standard DDS, however, ROS 2 also allows other two communication primitives, each with its own interface file format, which extend the communication services offered beyond the classic publish-subscribe paradigm. The communication primitives are:

- **Messages**: they are the most common and are used to exchange data packets among software modules in a publish-subscribe fashion; they are specified in `.msg` files. An illustrative scheme is provided in Figure 3.6, and the definition of the `Image` message from the `std_msgs` library is provided as an example in Listing 3.1.

- **Services**: they are used to request a computation to be performed by another software module and to receive the result, enabling a client-server, *stateless* communication; they are specified in `.srv` files. Note that in this context, the term *stateless* identifies a kind of communication that has no intermediate states between its beginning and end: in a service call, a *client* requests a computation to the *server*, which performs it and returns the result to the client, wihtout any further interaction between the two being possible and usually in a short time. This is useful for simple transmissions like quick parameter setting or

data retrieval, but not for complex interactions that require multiple steps, long computation times, or many possible outcomes. An illustrative scheme is provided in Figure 3.7, and the definition of the `AddTwoInts` service, which allows to perform the addition between two integers and is widely employed in tutorials, is provided as an example in Listing 3.2: note how the two request and result messages are delimited in the file by the three dashes.

- **Actions**: they are used to request a computation to be performed by another software module and to receive the result, enabling a client-server, *stateful* communication; they are specified in `.action` files. In this context, the term *stateful* identifies a kind of communication that has intermediate states between its beginning and end: in an action call, a *client* requests a computation to the *server*, which performs it and returns the result to the client, but the server can also send feedback to the client during the computation, and the client may even request the cancellation of the ongoing operation to the server. Moreover, the server may reply to the client at the end of the operation sending back two items: the actual output value of the computation, if any, and various codes that describe nearly any possible outcome. This is the most flexible and powerful communication primitive offered by ROS 2, usually employed in software modules that perform complex operations like navigation or tracking, where the client needs to know the status of the computation and possibly to interact with the server during its execution. An illustrative scheme is provided in Figure 3.8, and the definition of the `Fibonacci` action, which in many tutorials expresses the computation of the Fibonacci sequence up to a given order, is provided as an example in Listing 3.3: note how the three request, result, and feedback messages are again delimited in the file by the three dashes.

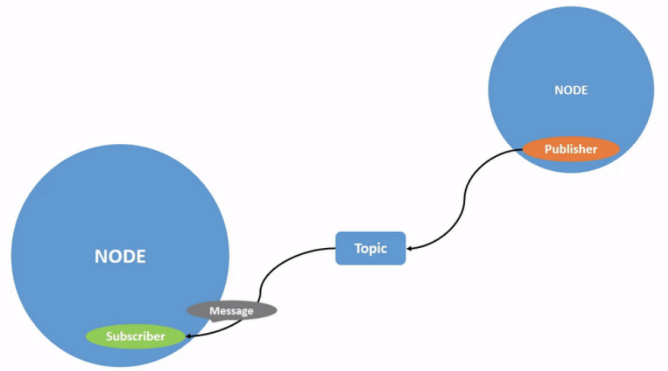

Figure 3.6: Scheme of the message communication primitive [33].

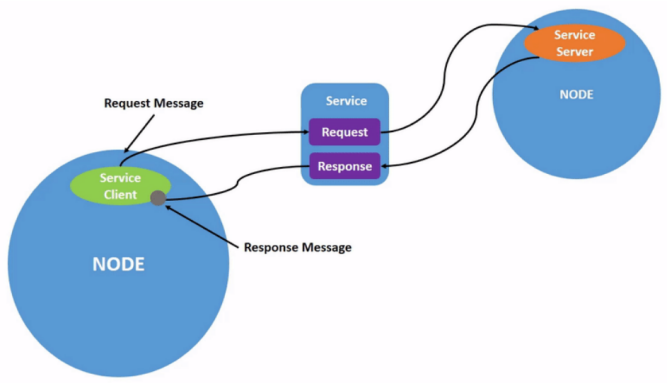

Figure 3.7: Scheme of the service communication primitive [34].

All of these are implemented on top of the RMW layer. In the case of DDS, they are all implemented using topics: a service maps to two topics, one for the request and one for the reply, while an action is implemented with two services: the *goal service* to place the request for a computation and the *result service* to request its output, and a *feedback topic*. In the case of other middleware that natively supports client-server communication, like Zenoh, the implementation is more straightforward and efficient, as the middleware itself manages the stateful communication and the client-server interaction, thus reducing the overhead and the complexity of the system.

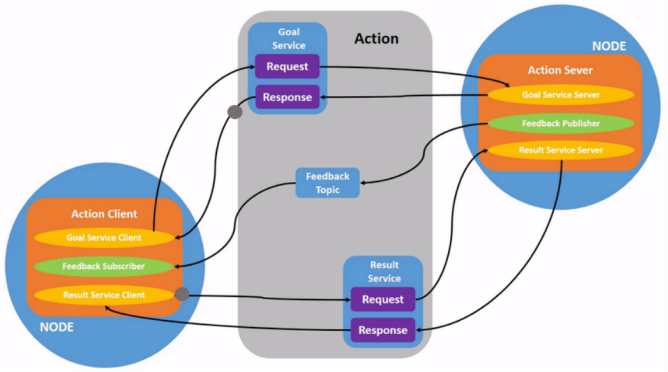

Figure 3.8: Scheme of the action communication primitive [35].

```
# This message contains an uncompressed image
# (0, 0) is at top-left corner of image
#

Header header # Header timestamp should be acquisition time of image
              # Header frame_id should be optical frame of camera
              # origin of frame should be optical center of camera
              # +x should point to the right in the image
              # +y should point down in the image
              # +z should point into to plane of the image
              # If the frame_id here and the frame_id of the CameraInfo
              # message associated with the image conflict
              # the behavior is undefined

uint32 height # image height, that is, number of rows
uint32 width  # image width, that is, number of columns

# The legal values for encoding are in file src/image_encodings.cpp

string encoding # Encoding of pixels -- channel meaning, ordering, size
                # taken from the list of strings in
                # include/sensor_msgs/image_encodings.h

uint8 is_bigendian # is this data bigendian?
uint32 step        # Full row length in bytes
uint8[] data       # actual matrix data, size is (step * rows)
```

Listing 3.1: Definition of the `sensor_msgs/Image` message.

```
int64 a
int64 b
---
int64 sum # = a + b
```

Listing 3.2: Definition of the `example_interfaces/AddTwoInts` service.

```
# Goal
int32 order
---
# Result
int32[] sequence # Variable-length array of 32-bit integers
---
# Feedback
int32[] sequence # Variable-length array of 32-bit integers
```

Listing 3.3: Definition of the `example_interfaces/Fibonacci` action.

Core API organization

To give an idea of how an architecture based on ROS for a distributed autonomous system may work, it is instrumental to present briefly the organization of the core API of ROS 2, which is shown in Figure 3.9. The core API is the set of libraries and tools that implement the basic functionalities offered by the ROS 2 middleware, intuitively starting from the communication layer and going up to the services offered to the application layer. Below the core API lies the communication middleware layer, which hosts the selected implementation(s) of the communication middleware(s) to employ; such implementations, when made available to the open-source community, are usually shipped together with ROS 2 base packages. This layer is represented in Figure 3.9 by the blue blocks at the bottom. On top of it all, there are code and applications that use the services offered by ROS 2 and everything else above them: these levels are represented by the top gray block in Figure 3.9. The core API, laying in

the middle, is divided into several modules, each of which is responsible for a specific aspect of the ROS 2 middleware. The most important levels are, from top to bottom:

- `rmw` **level**: the ROSMiddleWare level hosts a set of C libraries that abstract different implementations of communication middlewares like, *e.g.*, many DDS implementations and Zenoh, offering a common interface to upper levels to achieve basic communication functionalities.
- `rcl` **level**: the ROS Client Support Library level hosts a C library that implements basic entities like *nodes*, discussed in Section 3.1.3, *publishers* and *subscribers*, service and action *clients* and *servers*, and *timers*.
- **ROS Client Library level**: at this level, various libraries are implemented to offer support for programming languages other than C, like Python and C++. The entities defined in the lower levels are extended here, adding language-specific features and abstractions. Moreover, another core entity is defined at this level: the *executor*, which manages the execution of ROS-related jobs in dedicated OS threads, discussed in Section 3.1.3.

Nodes and executors

In ROS 2 applications, the most important entity is the *node*. Conceptually, a ROS 2 node is an operational unit of the distributed system, capable of performing a specific task for which it has been designed. In the code, irregardless of the programming language employed, a ROS 2 node is an object of a class, that usually expands the base `Node` class of the specific client library used, adding features that are specific for the operations that the node has to perform. Primarily, and thanks to class members and methods they inherit, nodes act as access points towards the middleware that enable

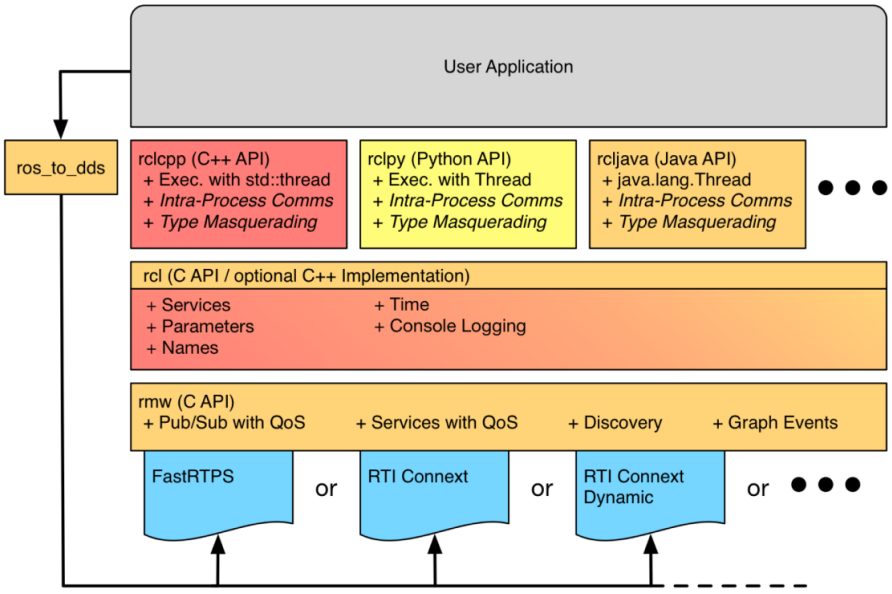

Figure 3.9: Core ROS 2 API organization [36].

application code to use ROS 2 features. For example, a node can create publishers and subscribers to exchange messages, service servers or clients, and even action servers and clients. Nodes have *parameters*, which are variables that can be set from the outside to configure the behavior of the node using a very versatile API: a node parameter is identified by a name and a type, and can be set at the startup of the node or at run time; in the context of an autonomous robot, a ROS 2 node parameter may be, *e.g.*, a gain for a control law being applied to the underlying physical system. Parameters can be queried and updated from both users and other software modules, *i.e.*, other nodes in the architecture, possibly deployed on different hosts in a distributed fashion.

The main idea is always that, to describe and design a distributed system, the node is a unit that performs computations that are aimed at performing a specific task, implementing a specific subsystem of the overall architecture. To achieve this, and to fully comply with the distributed paradigm even in real-time contexts, nodes generate computational *jobs* when specific events occur. Such events must be handled

accordingly, in a way that is orchestrated by the middleware. The situation at hand is described in Figure 3.10, which shows the event-based execution model of ROS 2. In this model, aside from synchronous computations that are part of the user code, part of the processing is handled asynchronously when the following specific events occur:

- a subscriber receives one or more messages;
- a timer expires;
- a service request is received;
- an action goal or result request is received;
- a parameter is updated.

Apart from timers, the parameter subsystem is implemented using services which map to topics together with actions, but for the sake of clarity Figure 3.10 highlights the most important event groups. Each of those can be processed by a user-specified callback, that is registered as part of the node configuration. Then, nodes carrying their asynchronous workload are added to an executor, which handles callbacks in groups over one or more OS thread; node that executors can handle multiple nodes at the same time. To set things in motion, hence the name, in any ROS 2 application the executor is then started by calling its `spin` method.

Over the years, the real-world performance of this paradigm has been thoroughly evaluated [37, 38, 39, 40]. What emerged, especially in the early versions of the framework, is that although conceptually correct it was affected by some design flaws. First of all, while in its ideal contexts the DDS performs well with little to no real-time overheads, the RMW abstraction could introduce some unwanted latencies. Moreover, the ROS 2 executor, which is the entity that manages the execution of the

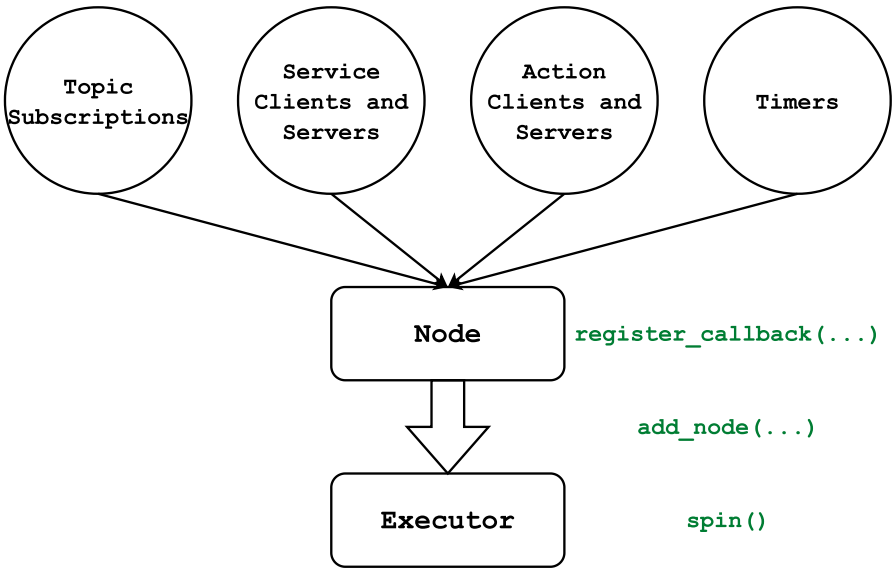

Figure 3.10: ROS 2 event-based execution model.

asynchronous jobs, was not designed to be real-time capable: its algorithm lacks any notion of priority and it seems to favor timers over any other kind of event, to empirically maximize responsiveness. If one thinks that commercial robots with ROS stacks are not only available but employed in the industry since almost a decade, it is clear that the ROS 2 executor is not a showstopper: in many contexts, things work fine because that level of strict real-time requirements is not needed, and as long as the system resources are not saturated both throughput and latencies remain acceptable even with such primitive algorithms. Nonetheless, the community has got the message: many solutions to these issues have been put in place and the core ROS 2 libraries listed in Section 3.1.3 are constantly updated and improved. The interested reader can find references to some of these solutions in [37], while a novel priority-based executor suited for real-time applications is presented in [41]. Finally, problems induced by the DDS, as discussed in Section 3.1.2, are addressed by the ongoing adoption of Zenoh as a communication middleware in ROS 2 [42], which is expected to be completed in the next few years. Another solution that deserves a mention is the *node composition* [36], which is similar to the way in which software modules are

handled in the MARTe2 framework presented in Section 2.3.4. In ROS 2, C++ nodes can be compiled in two ways: either as parts of executables or as standalone shared libraries[XII], denoted as *components*. Components can either be loaded and unloaded at run time, inside a process that hosts executors for their jobs acting as a *component container*, or they can be statically linked to an executable. The thing is that, in either of these ways, multiple nodes end up running inside the same process, thus sharing the same memory and address space. This allows to avoid the overhead of inter-process communication, completely bypassing the RMW layer and anything underneath, leading to very low latencies and high throughput. This is a very powerful feature that can be used to implement complex systems in an efficient way, as demonstrated in [36], provided that nodes are stable enough to run in the same process: in case one of them crashes, the whole process will be terminated, and all the other nodes will be stopped. This is a trade-off that must be carefully evaluated when designing a distributed system based on ROS 2, and the usual way of running each node in its own process remains the suggested one for the early stages of development and testing.

Launch system

A ROS 2 executable, or a group of such, must be launched using specific middleware commands to ensure that all necessary daemons and logging subsystems are activated. The middleware subsystem responsible for this is called the *launch system*, and the action of starting up multiple modules composing an architecture is called *launching*.

There are two ways to launch modules: either by using command-line tools to start each module individually, or by running scripts that specify the whole set of modules to run, together with their parameters and configuration options. These scripts are

[XII] A shared library is a collection of compiled functions and routines that can be used by multiple programs simultaneously, allowing for code reuse, efficient memory usage, and simpler software updates by sharing a single copy of the library in memory at runtime.

called *launch files*, and they are usually added as part of any package that contains executable code, and to any architecture once all the modules have been collected, to boot everything up with a single command. A launch file specifies, according to their format [43], the configuration with which the executable should be launched, including details ranging from input arguments to how logging should be managed, either in the console or in files, or both. Unless specified otherwise, any log files generated by the process output will be saved in a standard path, typically a subdirectory of the home directory (*e.g.*, `~/.ros/` on Linux).

Data recording and playback

For various purposes, such as testing or post-analysis, it may be necessary to capture all the messages that were handled by the middleware during a specific time interval and on particular topics. ROS 2 provides this service through a system called *bagging* [44]: using specific commands, it is possible to record messages sent on certain topics into a file, with configurable format and compression algorithms. This file, which also includes information about the message interfaces it contains, can be processed by other software to review the data. Bags can also be replayed from the beginning using another command, which creates ad-hoc publishers that will re-publish the recorded messages at an original or user-defined rate. This way, it is possible to develop, test, debug, and even tune algorithms offline, without the need to perform an experiment with the actual robot each time, just by replaying the recorded data, avoiding potentially risky or costly operations. This feature is so popular that the vendors of many proprietary scientific software like, *e.g.*, MATLAB [45] or LabVIEW [46], have developed plugins to read and write ROS 2 bags, and many open-source tools have been developed to parse and process them, some of which are part of the ROS 2 ecosystem themselves.

Visualization and simulation

A complete installation of ROS 2 includes several useful tools for debugging, such as commands for inspecting active topics, examining their interfaces, measuring the publication rate, and launching publishers as needed. There are also software tools that are not required for the operation of a robot and, in fact, are superfluous in that context but extremely useful during the development phase. It is appropriate to mention at least the following:

- **Gazebo** [47]: this is an open-source simulator natively compatible with ROS, capable of interacting with middleware topics to send and receive data. It allows for the creation of 3D models of robots—complete with sensors and actuators—as well as the environments in which they operate, using XML-based syntax, and provides a platform to test control and perception algorithms. Gazebo offers a 3D visualization of the simulated world, and it can be used to simulate the behavior of a robot in a virtual environment thanks to its embedded real-time physics engine. Moreover, thanks to its open-source nature, Gazebo can be extended with plugins that add new features or modify existing ones, like implement simulated sensors and scenario elements, and it can be integrated with other software tools like MATLAB [45], Simulink [48], or LabVIEW [46].

- **RViz** [49]: this is a 3D visualization tool that can be used to display the state of a robot and its environment, as well as the data that is exchanged among the various modules of the system. It is a very useful tool for debugging and testing, as it allows to visualize the data that the robot is processing in a way that is easy to understand, and to interact with the system by sending commands and receiving feedback. RViz can be used to display the robot's position and orientation, the data from its sensors, the map of the environment, and the

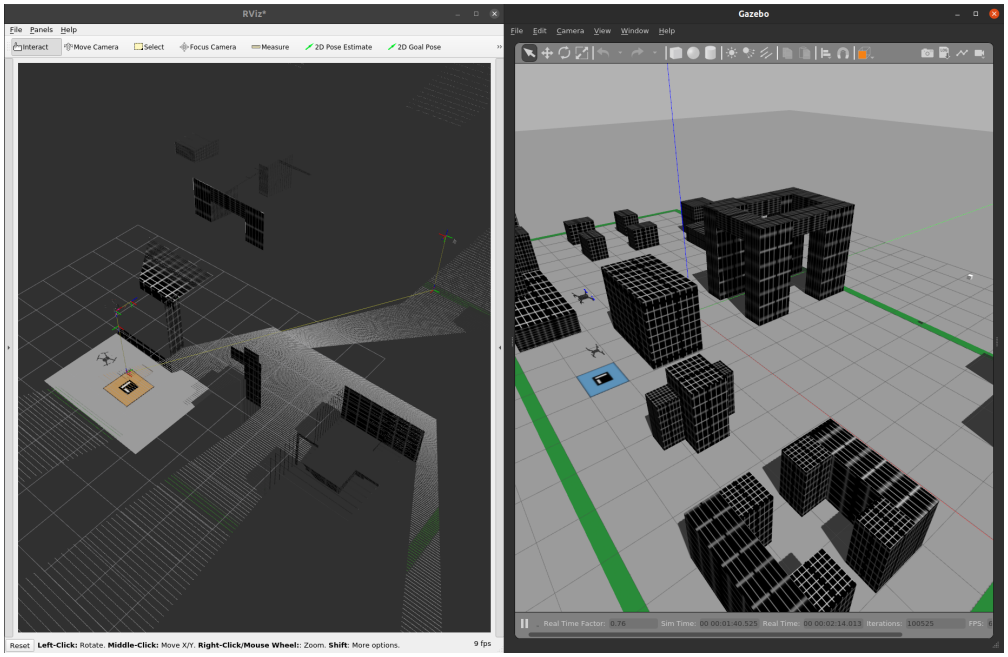

Figure 3.11: Simulation of an autonomous drone with imaging and depth sensors in Gazebo (right) and visualization of the data in RViz (left).

planned path, among other things. It can also be used to display the data exchanged among the various modules of the system, like the messages sent on the topics, the services called, and the actions performed. RViz can be used to visualize the data in 3D, 2D, or even in a tabular form, and it can be configured to display only the data that is relevant to the task at hand. As Gazebo, it is open-source and extensible with plugins that users can develop themselves.

An example of the capabilities of these two tools together is shown in Figure 3.11.

Applications and related frameworks

The previous sections highlighted the main features of the Robot Operating System 2 middleware, aimed at the development and organization of software for distributed autonomous robotic systems. The ROS 2 ecosystem is vast and varied, and it is used in many different application areas, from industrial automation to service robotics,

from autonomous vehicles to drones, from medical robotics to space exploration. The ROS 2 middleware is also used in many research projects, both in academia and in industry, and it is the basis for many open-source software libraries and tools that are widely used in the robotics community. Its features, discussed so far, clearly defined its relevance in the context of the present work, and it will be included in the proposed architecture as illustrated in the following Chapters. To conclude the presentation of ROS 2, some further references are in order to clarify its diffusion in both the research and industrial communities, as well as the variety of projects that are based on it.

One of the areas in which ROS has historically been employed the most is that of mobile, terrestrial robotics. In this field, there are a variety of problems to solve, ranging from localization to mapping, from path planning to obstacle avoidance, from perception to manipulation. One of the most famous projects related to this setting is Nav2 [50, 51, 52, 53, 54], the official ROS navigation stack for mobile robots. This package, originally developed to expose APIs for autonomous movement of mobile, planar robots, has evolved into a vast framework covering nearly every aspect of autonomous navigation: environment mapping and reconstruction, path planning and smoothing, obstacle avoidance, localization, and even human-robot interaction. A variety of algorithms for each case is available, and the system is designed to be modular and extensible in each of its parts, including visualization plugins for RViz and simulation elements for Gazebo. Operations carried out by the many modules of the framework are orchestrated by automata such as behavior trees [55].

Another area that robots revolutionized is that of industrial automation, in which manipulators of different sizes and shapes are employed to perform repetitive tasks with high precision and reliability. In this context, ROS 2 is used to control the robots and to interface them with other systems, like sensors and actuators, and to manage

the data that is exchanged among them. One of the most famous projects in this field is MoveIt [56, 57, 58, 59], the official ROS manipulation stack for industrial robots. As Nav2 for mobile robotics, MoveIt is very large in terms of functionalities it offers, all of which are built on ROS 2 tools and libraries. First, it allows the developer to define a physical and kinematic model of the robot using the Unified Robot Description Format (URDF), which is a particular kind of XML file in which the geometrical and dynamic properties of a robot can be specified: masses, inertias, joints, links, and even sensors. Then, it provides extensible software modules to interface with the robot hardware, which is usually made of custom microcontrollers with their own communication protocols and semantics. Finally, it allows the specification of states and movement operations thanks to many path planning and tracking algorithms, and it offers plugins for the RViz visualizer to turn it into a Graphical User Interface (GUI) for the robot.

One setting which is still relatively young is that of aerial robotics. Autonomous drones and aircraft can be used in a variety of applications thanks to their ability to fly, from surveillance to search and rescue, from agriculture to entertainment. The capabilities of the ROS 2 framework in this context are still being explored by the robotics community, particularly because a flying platform is more complex to control than a terrestrial one, and because the environment in which it operates is more dynamic and less predictable. Some preliminary results by the author, which led to the development of this work and can be considered its embryonic stage, are presented in [60, 61].

Finally, it is important to asses the kind of hardware platforms on which ROS 2 has been designed to run. If one recalls the discussions held in the previous Sections, it appers clear that the lowest tier of hardware on which ROS 2-based software can run

is a Single-Board Computer (SBC), which is usually the highest grade of hardware that can be found on autonomous robots. However, as previously remarked, robots are complex systems that usually also include specialized hardware like microcontrollers to drive both sensors and actuators. It would be desirable to be able to organize the code for such devices in a similar way to that based on ROS 2, even leveraging similar communication paradigms and abstractions. This is where the micro-ROS project [62] comes into play. Developed by eProsima, one of the major developers of DDS implementations as well as one of the greatest contributors to the ROS 2 codebase, micro-ROS is a reduced software stack implemented mostly in C, which offers a subset of the core ROS 2 functionalities with a minimal memory footprint and computational overhead. Its software stack, shown in Figure 3.12, is designed to run on a variety of Real-Time Operating Systems (RTOS) supporting their task scheduling and memory allocation APIs, and it is developed to preallocate or statically allocate memory to avoid dynamic memory allocation and deallocation, which is a common source of bugs and performance issues in real-time systems. The micro-ROS stack is designed to be used in conjunction with the full ROS 2 stack, exchanging data over topics in both directions thanks to a bridge put in place by the Micro XRCE-DDS [63] implementation, composed by an agent application running on a PC or SBC which talks with a client running on the microcontroller. Supported physical links include Ethernet, Wi-Fi, and even serial connections. As every solution presented up to this point, micro-ROS is open-source and its community is growing rapidly, as is the number of experimental evaluations, projects, and supported microcontrollers like, *e.g.*, the STM32 series from STMicroelectronics [64] or the ESP32 series from Espressif Systems [65].

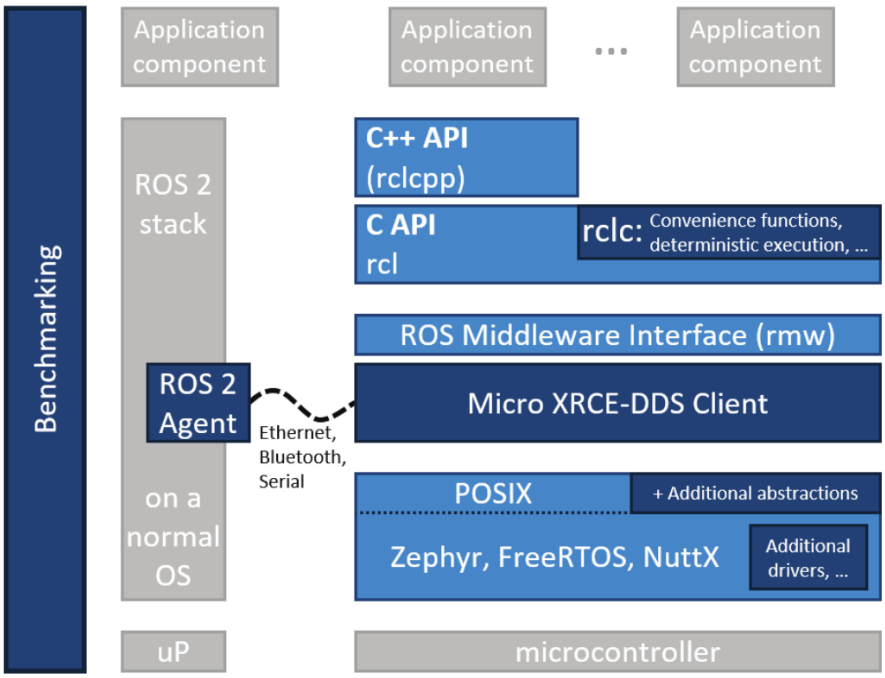

Figure 3.12: micro-ROS architecture [62].

3.2 Version control

The previous Sections introduced software solutions that solve specific issues concerning distributed systems, and more specifically, autonomous robots. The reasons why they are included in the architecture proposed in this work have been clarified, but applying the same criteria from Section 2.3 one can see how they still do not solve many problems, especially those explicitly related to the fundamental requirements presented in Section 2.2. In this Section and the next, two more software tools are introduced which clearly address all the requirements. Their inclusion, together with the solutions presented so far, will lead to the definition of the architecture that will be described in the following Chapters.

The first two requirements to address are the Modularity (M) and the Development (D) ones. In essence, the architecture must be modular in the sense that it must allow

for the development of software modules that can be easily maintained as standalone units, but at the same time it should provide a straightforward integration process when a larger-scale project, like a prototype, requires to set up many of these modules at the same time. Furthermore, when working in any of these two situations, the architecture should provide easy means of maintaining the single units composing a larger project and of tracking the changes made to them while working on the project itself; moreover, it should be possible to issue updates to these modules and propagate them to other projects that integrate them, in case they do not require a specific version of them. The technology that addresses these requirements is called *version control*.

Version control [66] is an essential element of contemporary software design and development, serving as a systematic mechanism for managing changes to source code, documentation, and other artifacts associated with a project over time. Fundamentally, version control involves the practice of tracking and managing modifications to software artifacts, thereby allowing developers to maintain a comprehensive history of changes, collaborate efficiently, and ensure consistent software quality. Version control systems (VCS) enable concurrent development by multiple contributors without conflicts, thus mitigating issues such as overwritten code, conflicting updates, and data loss. Furthermore, VCS facilitate the exploration of new features or bug fixes through branching, which isolates development efforts into independent streams, and merging, which integrates these branches into a cohesive codebase. This adaptability is crucial for modern software teams, who must concurrently manage multiple development lines, such as feature development, bug resolution, and experimental changes.

Branching constitutes a cornerstone of effective version control. It permits developers

to create independent versions of the codebase to work on distinct tasks without impacting the stability of the main code. These branches can be rigorously tested, reviewed, and subsequently merged into the primary branch, thereby minimizing the risk of introducing errors or destabilizing the main software version. Branching provides a structured framework for integrating new features and improvements, allowing developers to work independently until their contributions are complete and ready for integration. Additionally, version control systems include robust tools that simplify the merging of changes, even when modifications overlap across branches, thereby mitigating the complexities of integration conflicts.

The criticality of version control in software development is profound. It provides detailed tracking of every individual change made by team members, which facilitates an understanding of software evolution and captures the rationale behind each modification. This meticulous tracking becomes invaluable during debugging, as developers can analyze the precise changes that contributed to a malfunction and revert to a previous stable version if necessary. In large-scale software projects, where multiple developers are working simultaneously on a variety of features, a clear history of changes helps ensure accountability and transparency, both of which are vital for quality assurance and effective collaboration. The ability to revert changes or establish a timeline of modifications ensures that teams can efficiently correct mistakes without sacrificing valuable work or disrupting ongoing development processes.

Moreover, version control is indispensable for fostering team collaboration, especially in large projects involving numerous developers. It provides tools for managing and integrating the work of multiple contributors without overwriting critical changes or losing information. This facilitates an organized workflow, wherein each developer can work independently on assigned tasks while still contributing to the collective

project. Utilizing branches for distinct features or bug fixes ensures that various components of the codebase can progress in parallel, with integration occurring only after comprehensive review and testing. Such an approach prevents unintended side effects and guarantees that the main branch remains in a stable state.

Version control also plays a pivotal role in supporting automated processes such as continuous integration and continuous deployment (CI/CD). By maintaining an organized history of changes, version control systems enable automated testing and integration workflows, which ensure that modifications are validated before merging into the main branch. These workflows significantly contribute to software stability and reliability by catching issues early in the development lifecycle, thereby reducing the cost and complexity of fixing them later. Such automated processes are fundamental to modern agile development methodologies, where frequent, incremental updates are the standard, and maintaining software quality is paramount.

Ultimately, version control transcends the role of a mere tool for tracking changes—it is a foundational practice that underlies efficient, collaborative, and high-quality software development. By providing a structured framework for managing changes, version control empowers developers to innovate confidently, secure in the knowledge that their work is protected, reproducible, and reviewable. It fosters a disciplined approach to software development that enhances both individual productivity and team cohesion, making version control an indispensable component of the software engineering discipline.

For these reasons, it is clear that the adoption of a version control system as a fundamental element of the architecture would provide the means to satisfy the (M) and (D) requirements: as for the modularity requirement, the version control system would allow to maintain the software ranging from modules to larger projects as standalone

repositories under version control, eventually linked among each other. Furthermore, thanks to the branching and merging capabilities of the version control system, the development process would be streamlined, and the integration of the modules into a larger project would be facilitated. The version control system would also provide the means to track the changes made to the software, to issue updates to the modules, and to propagate them to other projects that integrate them, thus satisfying the development requirement. The inner workings of these processes are described in the following Section, which introduces the VCS of choice for the proposed architecture.

3.2.1 Git

Git [67] is one of the most influential version control systems in contemporary software development [68, 69], recognized for its robustness, efficiency, and distributed architecture. Its inception dates back to 2005, when Linus Torvalds, the creator of the Linux operating system [70], sought an effective and flexible version control system for managing the Linux kernel's development. At that time, the Linux kernel project was experiencing significant growth, with thousands of developers from around the globe contributing code. Initially, the project relied on a proprietary version control system called BitKeeper, which, although effective, became unsuitable for the open-source nature of the kernel due to licensing conflicts. This prompted Torvalds to develop Git, a system specifically tailored to meet the unique requirements of the kernel project, particularly distributed collaboration, speed, and support for non-linear workflows.

Linus Torvalds designed Git with several essential requirements in mind, derived from the practical challenges faced during the Linux kernel's rapid and distributed development. First, it needed to accommodate a distributed development model, whereby each developer could maintain a complete copy of the entire codebase, in-

cluding its full history. This decentralized approach provided resilience, ensuring that developers were not dependent on a central repository and could work independently without risking the loss of progress. The distributed nature also meant that individual contributors could experiment freely, with the assurance that any inadvertent changes would not compromise the project's integrity. The requirement for speed was equally paramount, as the system needed to handle the scale of contributions typical of the Linux project efficiently and support quick retrieval of project states. Furthermore, Torvalds emphasized the importance of strong support for non-linear development workflows, a feature critical for managing the extensive branching, merging, and contributions that characterize an open-source project of such magnitude and diversity.

Git's distributed architecture is one of its defining strengths, fundamentally distinguishing it from centralized version control systems [71]. In a centralized model, all developers rely on a single repository, which serves as the central point of authority for the project. In contrast, Git's distributed model allows each contributor to maintain a complete local clone of the repository, which includes the full history of all changes. This design not only provides a safety net, as the loss of a single repository does not impact the project's integrity, but also enhances productivity by allowing developers to work offline and synchronize changes when network access becomes available. In open-source projects like the Linux kernel, where developers are distributed across different geographies and time zones, the distributed model provides autonomy, enabling contributors to make progress independently while maintaining synchronization with the broader team. The distributed architecture also facilitates peer-to-peer collaboration, allowing developers to exchange changes directly without relying on a central authority, thereby promoting a more resilient and scalable development workflow.

Git's architecture and features make it a uniquely powerful and flexible version control system, which has revolutionized how teams approach software development. One significant advantage of Git is its use of snapshots rather than deltas. Unlike earlier version control systems, which typically stored differences between successive versions (deltas), Git creates a snapshot of the complete project state at each commit. This design choice ensures that the project's state can be easily and efficiently retrieved at any point in history, and it enhances data integrity through the use of cryptographic hashing. Each snapshot is uniquely identified using a hash function, which not only guarantees the integrity of the entire repository but also makes the history robust against corruption. This snapshot model forms the basis for the entire Git system, making retrieval, comparison, and merging of different states exceptionally efficient.

Git's branching and merging capabilities are also notable strengths, allowing developers to create branches with minimal overhead and merge them seamlessly when ready. Branches in Git are lightweight, which encourages experimentation without destabilizing the main codebase. This capability is particularly valuable for projects that require multiple, concurrent lines of development, such as new feature development, bug resolution, or exploratory work. The ability to create branches easily means that developers can work on different features or tasks simultaneously without fear of conflicting changes, which is vital in large-scale collaborative environments. Git's merging tools are designed to handle complex merging scenarios effectively, providing features such as conflict markers and automatic merging, which significantly reduce the complexity and risks associated with combining different branches of development.

Another advanced feature that Git offers, particularly beneficial in large and modular projects, is the concept of submodules. Submodules allow a Git repository to include

and track the content of other repositories, effectively enabling complex projects to integrate external dependencies while maintaining version control over them. Submodules are indispensable in managing dependencies, particularly when different components of a larger system are developed independently. By incorporating submodules, developers can ensure that specific versions of external libraries or components are consistently tracked, which guarantees compatibility and stability across different versions of the main project. This is especially useful when projects rely on third-party libraries that need to remain stable across multiple updates, preventing unforeseen compatibility issues that may arise from unsynchronized updates.

Submodules also play a key role in enabling modular development practices. Large projects often comprise multiple components that are logically separate but need to be version-controlled together. Submodules provide a means to maintain these components as distinct units, each with its own repository, while ensuring that they can be brought together in a cohesive manner within the parent project. This modularity is invaluable when different teams are responsible for different components, as each team can work independently on their respective modules, while the main project maintains coherence and consistency. Submodules thus foster a more manageable and scalable development process, particularly for projects that involve multiple dependencies with distinct development cycles. By leveraging submodules, teams can maintain a balance between modular autonomy and integrated consistency, ensuring that all parts of the system work harmoniously together while evolving independently.

The use of submodules extends beyond dependency management to foster more effective collaboration across different teams. When multiple teams work on separate but interrelated components, submodules provide an effective mechanism for versioning each component while maintaining an overarching structure. Each sub-

module is linked to a particular commit in its respective repository, which allows the main project to keep track of stable versions of dependencies. If updates are needed, the submodule can be updated to point to a new commit, ensuring that any changes are deliberate and controlled. This controlled mechanism ensures that all updates are tested before being integrated, which prevents disruptions to the main project's stability. Furthermore, submodules make it easier to reuse code across different projects, as they provide a clear and organized way to incorporate existing repositories without duplicating code, thereby fostering a more efficient and reusable development ecosystem.

A similar and complementary features to submodules is that of subtrees. Subtrees and submodules are both mechanisms for managing dependencies in a Git repository, but they offer distinct workflows and advantages. Subtrees allow developers to embed the content of an external repository into a subdirectory of the main repository. Unlike submodules, subtrees do not require additional configuration or extra repositories to be manually initialized when cloning the main repository. This makes subtrees more straightforward for new collaborators, as the embedded content becomes a natural part of the repository's history and structure, without any extra setup steps.

On the other hand, submodules are more suited for maintaining clear separations between distinct modules or components that need to evolve independently while remaining linked to a specific version in the main project. Submodules provide a finer level of control and facilitate updates across multiple repositories, which is particularly useful in larger projects where different teams are responsible for maintaining different components. Thus, while subtrees offer simplicity and convenience by integrating dependencies directly into the repository, submodules provide flexibility and modularity, enabling a cleaner separation of concerns.

These foundational features have led to Git becoming the de facto standard for version control in the software industry, trusted for both small-scale projects and large, highly distributed software systems. Git's flexibility, efficiency, and robust support for collaborative development have established it as an indispensable tool for modern software engineering, facilitating complex workflows and enabling distributed teams to innovate with confidence. Its influence extends beyond individual projects, shaping the entire software development landscape by enabling more effective collaboration, version management, and deployment practices. As software systems grow increasingly complex and teams become more distributed, the role of Git in managing the intricacies of software evolution remains pivotal, making it an essential element of any serious software development endeavor, as the one that this work proposes. To this end, the design of the architecture starts from the organization of every work it supports as a Git repository, as illustrated in the following Chapters. To fully satisfy the requirements (M) and (D), the architecture will also include support for both submodules and subtrees, as they are both useful in different contexts and for different purposes. While submodules are suited to allow for the combined integration and development of single software units into larger projects, they require a more complex setup and management, which is not always necessary. Subtrees, on the other hand, are more straightforward to use and are more suited in scenarios in which it is not expected that active development on the embedded content will be carried out, but rather that it will be used as a dependency and eventually only updated to newer versions. The architecture will be designed to support both mechanisms, allowing for the best of both worlds to be exploited.

3.3 Containerization

The previous Section highlighted the relevance of Git as a version control system in the context of this work, to provide a common ground for the organization and development of software modules. While Git is a powerful tool for managing source code, it does not completely address the Modularity (M) requirement: suppose that a software module requires external dependencies, *e.g.*, libraries or tools to be built and run. These external dependencies may not be made part of the module's repository but have to be set up in the systems that host it. Setting up a development and execution environment addressing situations like this is common in this line of work, but usually also quite a complex task. Multiply the complexities by the number of different hardware platforms that the software module has to run on, and you may get a set of problems that are hard to manage. This is where the Hardware (H) requirement comes into play: the architecture should provide a way to abstract the software modules from the hardware platforms they run on, to allow for the development of software that is portable across different platforms, and to simplify the setup of the development and execution environments. Historically, the technology that allowed this kind of abstraction is called *virtualization*, which in general comes at the price of a significant overhead in terms of computational resources and performance. Keep in mind that, in Section 2.2, the Overhead (O) requirement was also stated, which requires the architecture to be as lightweight as possible, to avoid any unnecessary computational overheads that may affect the performance of the software modules. The technology that addresses all these requirements, thus finalizing the foundational layer of the proposed architecture, is called *containerization*.

Containerization has emerged as a transformative paradigm in software deployment and infrastructure management, addressing a multitude of longstanding challenges

associated with portability, efficiency, and environmental consistency [72]. As software systems have grown increasingly complex, developers have faced considerable difficulties in ensuring that applications behave consistently across diverse machines and runtime environments. These challenges often arise from discrepancies in system configurations, dependencies, and library versions, giving rise to the notorious "it works on my machine" syndrome. Containerization effectively mitigates these issues by creating isolated environments that bundle an application with all of its necessary dependencies, ensuring consistent behavior across a variety of deployment contexts. This capability is of particular importance for modern, distributed software architectures, where consistency, rapid deployment, and scalability are essential components of operational success.

Containerization is often conflated with virtualization, but these technologies are fundamentally distinct in their architectures, use cases, and performance characteristics [73]. Virtualization relies on hypervisors to create virtual machines (VMs), each of which operates with its own guest operating system atop a host system. This approach provides a complete, independent virtual instance of a computer, encompassing its own operating system and resources, effectively emulating a full physical machine. However, the inclusion of a separate OS image for each VM introduces significant overhead, which can lead to suboptimal resource utilization—particularly when deploying numerous VMs on the same physical hardware. In contrast, containerization leverages the host operating system's kernel to create multiple isolated environments known as containers. Unlike virtual machines, containers share the host OS kernel while maintaining logical isolation at the process level. This design results in considerably less overhead, allowing for much higher resource density: many more containers can run concurrently on the same hardware compared to virtual machines. Consequently, containerization is preferable when the objective is to maximize resource efficiency

while maintaining isolated environments for different stages of development, testing, and deployment, without the need for complete OS-level isolation.

The origins of containerization can be traced back to early innovations in Unix-like operating systems. In the early 2000s, technologies such as FreeBSD Jails (2000) and Solaris Containers (2004) introduced the concept of partitioning a single operating system's resources into distinct, isolated instances. FreeBSD Jails enabled administrators to create secure, isolated environments capable of independently running services, while Solaris Containers provided an integrated method for managing multiple isolated application environments on a single instance of Solaris. Concurrently, the Linux ecosystem began to develop foundational features that would later form the bedrock of modern containerization technologies. Notably, Linux introduced kernel features such as cgroups (control groups) and namespaces—both essential to containerization's core principles of isolation and resource control. Linux namespaces provide process isolation, determining the visibility of system components like the filesystem, network interfaces, and other processes. In parallel, cgroups regulate the allocation of hardware resources—such as CPU, memory, and I/O—ensuring that containers do not monopolize or starve system resources, thus enabling fair resource sharing and predictable performance.

The introduction of these kernel-level features marked a pivotal advancement, laying the groundwork for the subsequent evolution of container management tools. Linux's cgroups and namespaces offered the first glimpse of what would eventually become a full-fledged container ecosystem, allowing developers to create environments where applications could operate independently while sharing the underlying system's resources efficiently. The development of these technologies represented a crucial step in enabling the practical implementation of containerization at scale, as it provided

the required process isolation and resource control that would later underpin all modern *container engines*.

Containerization's primary appeal lies in its inherent efficiency and flexibility. Unlike traditional virtual machines, containers do not incur the overhead of hosting multiple operating systems, leading to significantly faster start times, reduced resource consumption, and efficient utilization of hardware capabilities. These properties make containers ideal for microservices architectures, which decompose applications into smaller, independently deployable components. Containers offer an effective mechanism to encapsulate each microservice along with its respective dependencies, ensuring consistent operation regardless of the underlying infrastructure. This isolation also helps mitigate compatibility issues, allowing different microservices to use conflicting versions of dependencies without interference. The modularity offered by containerization enhances maintainability and promotes scalability, as individual microservices can be scaled independently according to demand.

The distinction between virtualization and containerization also has significant implications for operational strategies. Virtualization remains relevant in scenarios requiring full OS-level isolation, such as when multiple tenants need to be isolated from each other for security or compliance purposes. Virtual machines are effectively self-contained environments, providing an additional layer of abstraction that can be crucial in maintaining strict separation between different workloads. This makes virtualization an optimal choice for situations demanding rigid isolation or for supporting legacy applications that require their own dedicated operating environments. However, in scenarios where such complete isolation is not mandatory, containerization offers substantial advantages, including improved scalability, faster deployment times, and more efficient resource use. Containers are inherently more lightweight than VMs,

making them ideal for scenarios demanding rapid provisioning, horizontal scaling, and resource optimization. As a result, containerization has become the preferred solution for many use cases in software development, particularly those involving cloud-native applications, microservices, and environments requiring rapid scaling and adaptability.

Containerization represents a paradigm shift in how software is developed, deployed, and managed. By offering a lightweight, resource-efficient alternative to traditional virtualization, containerization maximizes resource utilization while maintaining isolated environments for reliable application execution. Although virtualization still plays a role in certain contexts, particularly where complete isolation is required, containerization has emerged as the favored approach for a wide range of use cases, offering unparalleled efficiency, portability, and flexibility. The progression of container technology highlights its critical role in supporting modern software infrastructure. It allows developers to achieve consistent and streamlined application deployment across any environment, driving the efficiency and scalability needed to meet the demands of contemporary software systems and distributed computing architectures.

In the context of this work, containerization can effectively help to build a common architectural ground that abstracts the software modules from the hardware platforms they run on, at the same time packaging with them every external and third-party dependency they require to be built and run. This way, the software modules can be developed and tested in a controlled environment, and then deployed on any hardware platform that supports the containerization technology, a condition that is not restrictive since today container engines are supported on nearly every commercial platform that the proposed solution may target. For this reason, modules and projects based on the proposed architecture will be managed through containers: the technical

details of this process are postponed to the next Chapters, where the architecture will be described in detail. To conclude this dissertation, the chosen containerization technology is introduced in the following Section.

3.3.1 Docker Engine

The release of Docker [74] in 2013 was a watershed moment in the evolution and widespread adoption of containerization [75, 76]. Docker unified the previously fragmented ecosystem of container technologies, incorporating existing Linux kernel features into a cohesive, user-friendly tool that dramatically simplified the process of container creation, deployment, and management. Docker's innovation lay in its ability to abstract the underlying complexity of containerization, providing a standardized interface and image format that made the technology accessible to a broader audience of developers and IT professionals. This accessibility facilitated a shift in how software was developed, tested, and deployed. Developers could now package their applications into lightweight, portable containers, ensuring consistent behavior across different environments, from local development machines to large-scale cloud deployments. Docker also contributed significantly to the establishment of container orchestration systems, which allow organizations to manage large numbers of containers seamlessly across clusters of machines.

The impact of Docker's introduction cannot be overstated; it fundamentally altered the software development lifecycle, enabling teams to deploy applications with unprecedented speed and consistency. By providing developers with an easy-to-use toolset, Docker democratized the use of container technology, which had previously been accessible only to those with in-depth systems expertise. Docker images standardized the definition of application environments, reducing the configuration drift

that plagued software deployment efforts. Docker Hub [77], an associated public *registry* of container *images*, further accelerated adoption by providing a central repository from which developers could share and reuse container images, promoting community collaboration and reducing redundancy. Moreover, since its birth as a Linux-only tool, Docker has evolved to support also the Windows and macOS operating systems through its Docker Desktop release [78], although with a limited set of features available on these platforms, which also require a thin virtualization layer to work. This revolution has also impacted the research world outside of the cloud computing and software development industries, as it has been adopted in many scientific fields to manage the software tools and libraries that are used to analyze data and run simulations, as well as to distribute the results of the research and novel algorithms and procedures in a way that ensures reproducibility, transparency, and accessibility [79].

The first key component of Docker's utility is the Dockerfile, which serves as the blueprint for creating Docker images. A Dockerfile is a simple text file that contains a series of instructions on how to assemble a particular Docker image. These instructions include the base image to use, any software dependencies to be installed, environmental variables, and commands to be executed when building the image. When a Dockerfile is processed by the Docker engine, it creates a Docker image, which is a read-only template containing the application and its environment. Docker images can then be instantiated as running containers. Containers, therefore, are the execution units created from Docker images, providing a live instance of the encapsulated application. This workflow is at the core of Docker's portability and consistency, ensuring that the entire environment is replicated across different systems with high fidelity.

Another fundamental feature of Docker is its use of union filesystems [80], which are specialized filesystems that merge multiple layers into a single coherent view. Union filesystems are particularly useful in containerization as they allow multiple layers to be stacked, where each layer represents a set of *filesystem changes*. This approach enables efficient storage as new layers can be added or modified without altering the underlying base layers, thereby conserving disk space and allowing for rapid image creation and reuse. Specifically, Docker employs OverlayFS by default to create image layers [81]. Docker images are built in layers, with each layer representing a set of filesystem changes such as installing a package or adding a configuration file. The union filesystem allows Docker to stack these layers on top of each other, presenting them as a single coherent filesystem. This approach provides significant efficiencies in both storage and resource usage, as multiple images can share the same underlying layers. For example, if multiple images are based on the same base operating system, Docker stores only one copy of that base layer. OverlayFS, for example, enables efficient management of these layers by allowing new changes to be made in an uppermost layer without altering the lower layers, enforcing a copy-on-write mechanism[XIII]. This layered approach not only conserves disk space but also accelerates the building of new images by reusing existing layers.

Docker Compose [82] is another powerful tool within the Docker ecosystem that facilitates the management of multi-container applications. With Docker Compose, developers can define and run multi-container Docker applications using a YAML[XIV] file to configure the services, networks, and volumes required by the application. This capability is particularly useful for orchestrating the deployment of complex

[XIII] In this context, *copy-on-write* denotes a paradigm according to which stored data is shared among multiple entities keeping a single copy, until one of such entities must modify it; in this case, data is actually copied and a new instance is assigned to such entity that can now modify it.

[XIV] Yet Another Markup Language, a popular markup language for configuration and text files.

applications that require multiple components—such as databases, backend services, and front-end web servers—to work together seamlessly. Docker Compose allows these components to be defined as individual services, each in its own container, and specifies how they interact. This simplifies the process of setting up development environments and ensures that the entire application stack can be easily replicated and deployed.

Another crucial feature of Docker are *volumes* and *volume mounts*, which are of paramount importance for data persistence. Containers are ephemeral by nature: when a container is stopped or deleted, all data within it is lost. To overcome this limitation, Docker uses volumes, which are directories or files stored outside the container's writable layer. Volumes can be mounted into one or more containers, allowing data to persist even if a container is removed or updated. This capability is essential for stateful applications, such as databases, which require durable storage that outlives the individual container instances. By using volume mounts, developers can also share data between containers, enabling efficient communication and data exchange between different parts of an application or of a distributed software architecture.

Networking is another key area where Docker provides significant capabilities, enabling containers to communicate with each other and with the outside world. The networking model of Docker includes several types of networks hereby only simply mentioned such as *bridge*, *host*, and *overlay*, each suited to different use cases. Bridge networks are the default and are typically used to enable communication between containers running on the same host. Host networking, on the other hand, allows a container to share the network stack of the host, providing greater network performance and simplifying access to services running on the host. This mode is particularly useful in scenarios where the containerized service needs to avoid the

overhead of NAT[XV] and requires direct access to the network interfaces available on the host.

Docker also provides mechanisms to control the resources allocated to containers, such as CPU and RAM using, *e.g.*, cgroups on Linux. Resource limiting is critical to ensure that no single container can consume disproportionate system resources, which could negatively affect other containers running on the same host. Docker allows users to specify limits on the CPU shares and memory usage for each container, ensuring predictable performance and efficient utilization of system resources. By configuring these limits, administrators can create a balanced environment where all containers receive their fair share of resources, preventing resource contention and ensuring stable operation. This capability extends, through additional tools, to the management of additional dedicated processing hardware like GPUs[XVI], which can be shared among containers in a controlled way.

Finally, Docker can use the Linux capabilities subsystem to manage hardware and system call access in a granular manner. Instead of giving containers full root privileges, which could pose security risks, Docker allows specific capabilities to be granted as needed. For instance, a container may need the capability to bind to low-numbered network ports or change the system time, but it does not need broader privileges that could further compromise the host. By selectively enabling only the necessary capabilities, Docker minimizes the attack surface of containers, enhancing security while maintaining the required functionality for each application.

This Section introduced and described the Docker Engine containerization solution alongside many of its features. The choice of the functionalities to discuss here was

[XV] Network Address Translation.

[XVI] Graphics Processing Unit, specialized high-power processors originally developed for videogames and now commonly used for parallel workloads in many industrial and scientific applications.

not coincidental: each of the features described is essential to the architecture, to fully employ Docker as the solution that addresses the Hardware (H) requirement. The Docker Engine is used in the present work to build and manage hardware and system abstraction layers that establish a common environment for both development and execution of software modules and projects among all the platforms that are of interest in the contexts covered by this work. The following Chapters will explain their implementation in detail, showing how the architecture is built on top of these technologies to satisfy all the requirements stated in Section 2.2.

Bibliography

[1] E. S. Raymond. *How To Become A Hacker*. 2001. URL: http://www.catb.org/esr/faqs/hacker-howto.html (visited on 11/04/2024).

[2] T. Fischer et al. "A RoboStack Tutorial: Using the Robot Operating System Alongside the Conda and Jupyter Data Science Ecosystems". In: *IEEE Robotics & Automation Magazine* 29.2 (June 2022), pp. 65–74.

[3] Anaconda Inc. *Conda*. URL: https://docs.conda.io/projects/conda/en/stable/ (visited on 11/06/2024).

[4] eProsima. *Vulcanexus, the All-in-One ROS 2 tool set*. URL: https://vulcanexus.org/ (visited on 11/06/2024).

[5] R. L. Bocchino et al. "F Prime: An Open-Source Framework for Small-Scale Flight Software Systems". In: *Proc. of the 2018 Small Satellite Conference*. Aug. 2018.

[6] A. C. Neto et al. "MARTe: A Multiplatform Real-Time Framework". In: *IEEE Transactions on Nuclear Science* 57.2 (Apr. 2010), pp. 479–486.

[7] P. Naur and B. Randell. *Software Engineering: Report of a conference sponsored by the NATO Science Committee, Garmisch, Germany, 7th-11th October 1968*. Tech. rep. NATO Scientific Affairs Division, 1969.

[8] G. Pardo-Castellote. "OMG Data-Distribution Service: architectural overview". In: *Proc. of the 23rd International Conference on Distributed Computing Systems*. May 2003, pp. 200–206.

[9] J. M. Schlesselman, G. Pardo-Castellote, and B. Farabaugh. "OMG data-distribution service (DDS): architectural update". In: *Proc. of the 2004 IEEE Military Communications Conference (MILCOM)*. Vol. 2. Oct. 2004, pp. 961–967.

[10] M. S. Essers and T. H. J. Vaneker. “Evaluating a Data Distribution Service System for Dynamic Manufacturing Environments: A Case Study”. In: *Procedia Technology*. 2nd International Conference on System-Integrated Intelligence: Challenges for Product and Production Engineering 15 (Jan. 2014), pp. 621–630.

[11] S. Saxena, H. E. Z. Farag, and N. El-Taweel. “A distributed communication framework for smart Grid control applications based on data distribution service”. In: *Electric Power Systems Research* 201 (Dec. 2021), p. 107547.

[12] A. Corsaro et al. “Quality of service in publish/subscribe middleware”. In: *Global Data Management* 19.20 (2006), pp. 1–22.

[13] eProsima. *1.1. What is DDS? — Fast DDS 3.1.0 documentation*. URL: https://fastdds.docs.eprosima.com/en/latest/fastdds/getting_started/definitions.html (visited on 11/13/2024).

[14] A. Corsaro et al. “Zenoh: Unifying Communication, Storage and Computation from the Cloud to the Microcontroller”. In: *2023 26th Euromicro Conference on Digital System Design (DSD)*. Sept. 2023, pp. 422–428.

[15] G. Baldoni et al. “Facilitating distributed data-flow programming with Eclipse Zenoh: the ERDOS case”. In: *Proc. of the 1st Workshop on Serverless mobile networking for 6G Communications*. Association for Computing Machinery, July 2021, pp. 13–18.

[16] S. Greising and A. Reichert. “Zenoh Whitepaper”. FLECS Technologies GmbH, Jan. 2023.

[17] S. Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (May 2022), eabm6074.

[18] M. Quigley et al. “ROS: an open-source Robot Operating System”. In: *ICRA workshop on open source software*. Vol. 3. May 2009.

[19] Open Robotics. *Installation — ROS 2 Documentation: Rolling documentation*. URL: https://docs.ros.org/en/rolling/Installation.html (visited on 11/12/2024).

[20] Canonical. *Ubuntu Linux*. URL: https://ubuntu.com/ (visited on 11/12/2024).

[21] Open Robotics. *Open Robotics*. URL: https://www.openrobotics.org (visited on 11/12/2024).

[22] B. Gerkey. *Why ROS 2?* June 2014. URL: https://design.ros2.org/articles/why_ros2.html (visited on 11/13/2024).

[23] D. Thomas. *Changes betweek ROS 1 and ROS 2*. Sept. 2015. URL: https://design.ros2.org/articles/changes.html (visited on 11/13/2024).

[24] W. Woodall. *ROS on DDS*. June 2014. URL: https://design.ros2.org/articles/ros_on_dds.html (visited on 11/13/2024).

[25] Open Robotics. *ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/index.html (visited on 11/13/2024).

[26] Open Robotics. *Tutorials — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials.html (visited on 11/13/2024).

[27] D. Thomas. *A universal build tool*. Mar. 2017. URL: https://design.ros2.org/articles/build_tool.html (visited on 11/13/2024).

[28] D. Thomas. *colcon - collective construction*. URL: https://colcon.readthedocs.io/en/released/ (visited on 11/13/2024).

[29] kitware. *CMake*. URL: https://cmake.org/ (visited on 11/13/2024).

[30] P. Eby et al. *Setuptools*. URL: https://setuptools.pypa.io/en/latest/userguide/ (visited on 11/13/2024).

[31] D. Thomas. *The build system "ament_cmake" and the meta build tool "ament_tools"*. July 2015. URL: https://design.ros2.org/articles/ament.html (visited on 11/13/2024).

[32] D. Thomas. *Interface definition using .msg / .srv / .action files*. Mar. 2019. URL: https://design.ros2.org/articles/legacy_interface_definition.html (visited on 11/13/2024).

[33] Open Robotics. *Understanding topics — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html (visited on 11/13/2024).

[34] Open Robotics. *Understanding services — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html (visited on 11/13/2024).

[35] Open Robotics. *Understanding actions — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html (visited on 11/13/2024).

[36] S. Macenski et al. "Impact of ROS 2 Node Composition in Robotic Systems". In: *IEEE Robotics and Automation Letters* 8.7 (July 2023), pp. 3996–4003.

[37] C. Bédard et al. “Message flow analysis with complex causal links for distributed ROS 2 systems”. In: *Robotics and Autonomous Systems* 161 (Mar. 2023), p. 104361.

[38] Yuya Maruyama, S. Kato, and T. Azumi. “Exploring the performance of ROS2”. In: *2016 International Conference on Embedded Software (EMSOFT)*. Oct. 2016, pp. 1–10.

[39] L. Puck et al. “Distributed and Synchronized Setup towards Real-Time Robotic Control using ROS2 on Linux”. In: *2020 IEEE 16th International Conference on Automation Science and Engineering (CASE)*. Aug. 2020, pp. 1287–1293.

[40] T. Kronauer et al. “Latency Analysis of ROS2 Multi-Node Systems”. In: *2021 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*. Sept. 2021, pp. 1–7.

[41] J. Staschulat, I. Lutkebohle, and R. Lange. “The rclc Executor: Domain-specific deterministic scheduling mechanisms for ROS applications on microcontrollers: work-in-progress”. In: *Proc. of the 2020 International Conference on Embedded Software (EMSOFT)*. IEEE, Sept. 2020, pp. 18–19.

[42] Open Robotics. *ROS 2 Alternative middleware report*. Tech. rep. Sept. 2023. URL: https://discourse.ros.org/t/ros-2-alternative-middleware-report/33771 (visited on 11/13/2024).

[43] Open Robotics. *Launch — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Launch/Launch-Main.html (visited on 11/13/2024).

[44] Open Robotics. *Recording and playing back data — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html (visited on 11/13/2024).

[45] MathWorks. *MATLAB*. URL: https://www.mathworks.com/products/matlab.html (visited on 11/13/2024).

[46] National Instruments. *LabVIEW*. URL: https://www.ni.com/en/shop/labview.html (visited on 11/13/2024).

[47] Open Robotics. *Gazebo*. URL: https://gazebosim.org/home (visited on 11/13/2024).

[48] MathWorks. *Simulink*. URL: https://www.mathworks.com/products/simulink.html (visited on 11/13/2024).

[49] Open Robotics. *RViz*. URL: https://github.com/ros2/rviz (visited on 11/13/2024).

[50] Open Navigation. *Nav2*. URL: https://docs.nav2.org/ (visited on 11/13/2024).

[51] S. Macenski et al. "From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2". In: *Robotics and Autonomous Systems* 168 (Oct. 2023), p. 104493.

[52] S. Macenski et al. "The Marathon 2: A Navigation System". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Oct. 2020, pp. 2718–2725.

[53] S. Macenski et al. "Regulated pure pursuit for robot path tracking". In: *Autonomous Robots* 47.6 (Aug. 2023), pp. 685–694.

[54] A. Merzlyakov and S. Macenski. "A Comparison of Modern General-Purpose Visual SLAM Approaches". In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Sept. 2021, pp. 9190–9197.

[55] M. Iovino et al. "A survey of Behavior Trees in robotics and AI". In: *Robotics and Autonomous Systems* 154 (2022), p. 104096.

[56] PickNik Robotics. *MoveIt 2*. URL: https://moveit.picknik.ai/main/index.html (visited on 11/13/2024).

[57] Michael Görner et al. "MoveIt! Task Constructor for Task-Level Motion Planning". In: *2019 International Conference on Robotics and Automation (ICRA)*. ISSN: 2577-087X. May 2019, pp. 190–196. DOI: 10.1109/ICRA.2019.8793898. URL: https://ieeexplore.ieee.org/abstract/document/8793898 (visited on 11/12/2024).

[58] D. T. Coleman et al. "Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study". In: *Journal of Software Engineering for Robotics* 5.1 (May 2014), pp. 3–16.

[59] A. Bonci et al. "Robot Operating System 2 (ROS2)-Based Frameworks for Increasing Robot Autonomy: A Survey". In: *Applied Sciences* 13.23 (Jan. 2023), p. 12796.

[60] L. Bianchi et al. "A novel distributed architecture for unmanned aircraft systems based on Robot Operating System 2". In: *IET Cyber-Systems and Robotics* 5.1 (2023), e12083.

[61] L. Bianchi et al. “Efficient visual sensor fusion for autonomous agents”. In: *Proc. of the 7th International Conference on Control, Automation and Diagnosis (ICCAD)*. IEEE, May 2023, pp. 01–06.

[62] eProsima. *micro-ROS*. URL: https://micro.ros.org/ (visited on 11/13/2024).

[63] eProsima. *Micro XRCE-DDS*. URL: https://micro-xrce-dds.docs.eprosima.com/en/latest/ (visited on 11/13/2024).

[64] STMicroelectronics. *STM32 Microcontrollers*. URL: https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html (visited on 11/13/2024).

[65] Espressif Systems. *ESP32 Wi-Fi & Bluetooth SoC*. URL: https://www.espressif.com/en/products/socs/esp32 (visited on 11/13/2024).

[66] N. N. Zolkifli, A. Ngah, and A. Deraman. “Version Control System: A Review”. In: *Procedia Computer Science* 135 (Jan. 2018), pp. 408–415.

[67] L. Torvalds et al. *Git*. URL: https://git-scm.com/ (visited on 11/14/2024).

[68] J. Loeliger and M. McCullough. *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development*. "O’Reilly Media, Inc.", Aug. 2012.

[69] C. Bird et al. “The promises and perils of mining git”. In: *2009 6th IEEE International Working Conference on Mining Software Repositories*. May 2009, pp. 1–10.

[70] L. Torvalds et al. *Linux kernel*. URL: https://kernel.org/ (visited on 11/14/2024).

[71] C. Rodríguez-Bustos and J. Aponte. “How Distributed Version Control Systems impact open source software projects”. In: *2012 9th IEEE Working Conference on Mining Software Repositories (MSR)*. June 2012, pp. 36–39.

[72] C. Pahl et al. “Cloud Container Technologies: A State-of-the-Art Review”. In: *IEEE Transactions on Cloud Computing* 7.3 (July 2019), pp. 677–692.

[73] R. Dua, A R. Raja, and D. Kakadia. “Virtualization vs Containerization to Support PaaS”. In: *2014 IEEE International Conference on Cloud Engineering*. Mar. 2014, pp. 610–614.

[74] Docker Inc. *Docker*. URL: https://www.docker.com/ (visited on 11/14/2024).

[75] D. Merkel. “Docker: lightweight Linux containers for consistent development and deployment”. In: *Linux Journal* 2014.239 (May 2014), 2:2. URL: https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment (visited on 11/14/2024).

[76] J. Turnbull. *The Docker Book: Containerization Is the New Virtualization*. James Turnbull, July 2014.

[77] Docker Inc. *Docker Hub Container Image Library*. URL: https://hub.docker.com (visited on 11/14/2024).

[78] Docker Inc. *Docker Desktop*. URL: https://www.docker.com/products/docker-desktop/ (visited on 11/14/2024).

[79] C. Boettiger. “An introduction to Docker for reproducible research”. In: *SIGOPS Operating Systems Review* 49.1 (Jan. 2015), pp. 71–79.

[80] R. Dua et al. “Performance analysis of Union and CoW File Systems with Docker”. In: *2016 International Conference on Computing, Analytics and Security Trends (CAST)*. Dec. 2016, pp. 550–555.

[81] N. Mizusawa, K. Nakazima, and S. Yamaguchi. “Performance Evaluation of File Operations on OverlayFS”. In: *2017 Fifth International Symposium on Computing and Networking (CANDAR)*. Nov. 2017, pp. 597–599.

[82] Docker Inc. *Docker Compose*. URL: https://github.com/docker/compose (visited on 11/14/2024).